The Critical Line Algorithm

A parametric active-set method for the constrained mean–variance efficient frontier, with a LARS/LASSO twin, as implemented in `cvxcla`

Thomas Schmelzer
Jebel Quant Research, Abu Dhabi

June 20, 2026

Abstract

`cvxcla` is an open implementation of the Critical Line Algorithm (CLA) of Markowitz, which computes the *entire* mean–variance efficient frontier of a portfolio problem exactly, rather than sampling it point by point, under box bounds together with general linear equality ($Aw = b$) and inequality ($Gw \leq h$) constraints. The frontier is piecewise linear in weight space: between two consecutive *turning points* the optimal weights are an affine function $w(\lambda) = \alpha + \lambda\beta$ of a single scalar parameter λ that trades expected return against variance, and the algorithm walks from the maximum-return portfolio ($\lambda = \infty$) to the minimum-variance portfolio ($\lambda = 0$), changing the status of exactly one variable or one active constraint at each turning point, each step a single block-eliminated Karush–Kuhn–Tucker solve.

The frontier is Markowitz's; the contribution here is threefold. First, an *operator interface*: the turning-point loop reaches the covariance only through a few linear-algebraic operations, so capturing that contract as a small protocol decouples the algorithm from the covariance representation and lets structured risk models (diagonal plus low rank) trace the same exact frontier without ever materialising a dense $n \times n$ matrix, at a fraction of the cost when their structure is genuinely low-rank. Second, a *unification*: the CLA is the one-parameter specialisation of multi-parametric quadratic programming and shares its mechanics with the LARS/LASSO homotopy of statistical learning, so the same path-tracing engine, carrying the same general linear constraints, returns the exact constrained efficient frontier and the exact constrained LASSO regularisation path alike. Third, an *open, tested, and benchmarked implementation*, with event logic hardened against degeneracy and floating-point noise.

This hardening is bounded, not total: near-degenerate problems are repaired by projecting sub-tolerance round-off back onto the feasible set, while genuinely rank-deficient or degenerate instances are declined with an actionable diagnosis rather than mis-solved (the boundaries, including the unproven worst case of the lowest-index tie-break, are stated precisely in the body). The benchmarks should be read in this light: to the best of our knowledge no maintained, installable large-scale CLA implementation is available to compare against, so the runtime comparison is against PyPortfolioOpt (the most widely-used open CLA) together with internal controls, not a tuned large-scale competitor.

Contents

1 Introduction

The mean–variance portfolio selection problem was introduced by Markowitz (1952) in doctoral work completed at the University of Chicago at the age of 24; Markowitz later recounted that Milton Friedman, on his dissertation committee, questioned whether the result was economics at all. The problem asks for the allocation of capital across n assets that minimises risk for a given level of expected return. Sweeping the return level produces a one-parameter family of optimal portfolios

whose risk–return image is the *efficient frontier*. Four years later, at RAND, Markowitz gave the problem its constructive solution: the Critical Line Algorithm (CLA), introduced by Markowitz (1956) and developed in full in his 1959 monograph (Markowitz, 1959). The CLA computes this frontier *exactly and in full*: it returns a finite list of *turning points* between which the frontier is described in closed form, so no discretisation or repeated re-optimisation is required. It belongs to a broad family of parametric quadratic-programming and active-set methods for the efficient frontier (Wolfe, 1959; Perold, 1984; Niedermayer and Niedermayer, 2010; Bailey and López de Prado, 2013); Markowitz and Todd (2000) give the canonical description and a reference implementation, and Markowitz et al. (2021) a recent step-by-step review that also extends the method to mean–semivariance, while Perold (1984), Stein et al. (2008) and Hirschberger et al. (2010) develop large-scale and active-set variants.

When Markowitz designed the CLA, the structured covariance models that dominate modern portfolio construction did not yet exist; classical implementations therefore assume a dense matrix. Structure arrived with the single-index model of Sharpe (1963), developed at Markowitz's suggestion to tame the estimation and computational burden of a full covariance by writing it as a diagonal matrix plus a single rank-one term. Modern multi-factor risk models generalise this diagonal-plus-low-rank form, and it is precisely this structure that the operator abstraction of this paper exploits: the covariance the classical CLA treats as dense is, in practice, almost never dense.

We claim no new turning points (the frontier is Markowitz's, and `cvxcla` traces exactly the same one) but a new *interface* to it: we capture the covariance contract as three operations (a matrix–vector product, a free-block solve, and a free-to-blocked cross product) so that structured risk models trace the identical frontier without touching the algorithm. The one element here that is, to our knowledge, not standard in existing CLA implementations (which operate on an explicit dense covariance (Markowitz and Todd, 2000; Bailey and López de Prado, 2013)) is this *covariance-operator abstraction*. The turning-point loop touches the covariance through only three operations: a matrix–vector product, a solve against the free-asset block, and a free-to-blocked cross product. Capturing exactly this contract as a small protocol decouples the algorithm from the covariance representation, so a structured risk model (e.g. a diagonal-plus-low-rank factor covariance solved through the Woodbury identity) drops in as a backend and traces the *same exact frontier* at lower memory in every case, and at a fraction of the time when its rank is genuinely low (few factors over many assets), without a single change to the turning-point loop. This is the paper's main engineering contribution; the experiments of §9 quantify both the memory saving and the regime-dependent speed-up.

Contributions. The turning points are Markowitz's; the contribution here is one of *interface, unification, and a reusable implementation*, organised under three headings:

1. **An operator interface.** A covariance-operator abstraction (§5) reduces the covariance contract to three operations (a matrix–vector product, a free-block solve, and a free-to-blocked cross product), decoupling the turning-point loop from the covariance representation; to our knowledge this abstraction is not standard in existing CLA implementations. It is instantiated by a diagonal-plus-low-rank factor backend (§5) that solves the free block through the Woodbury identity, never materialising an $n \times n$ matrix, and traces the identical frontier with sub-quadratic empirical scaling (§9.4).

2. **A unification.** We make explicit that the CLA is one instance of a broader parametric active-set family: the one-parameter specialisation of multi-parametric quadratic programming and explicit model predictive control, and the mean–variance counterpart of the LARS/LASSO

homotopy of statistical learning (§10), a correspondence the `GramCovariance` backend makes literal.

3. **An open, tested, benchmarked implementation.** Hardened event logic, a Bland-style lowest-index tie-break (§4.4) and a floating-point slope floor (§4.3), keeps the trace deterministic and prevents cycling and out-of-bounds drift on the degenerate cases we test; reproducible benchmarks (§9) isolate the genuinely algorithmic speed-up (the structured solve) from incidental linear-algebra bookkeeping.

Underpinning all three, the turning-point loop is written for general constraints: box bounds together with an arbitrary linear equality system $Aw = b$ and arbitrary linear inequalities $Gw \leq h$ (a budget, sector or factor neutralities, group or sector exposure caps), not merely the single budget row of the classical CLA. An active inequality row is carried through the same reduced solve as an extra equality row (§10), so this generality costs the box-only problem nothing. We do *not* claim the fastest dense large-scale solve (§8); near-degenerate problems are handled by projecting the sub-tolerance round-off they produce back onto the feasible set (§9.6), while two cases are declined with an actionable diagnosis rather than mis-solved: a numerically rank-deficient free block (§9.6), and a degenerate maximum-return vertex under a general equality system, where the free set is too small to span the constraints (§8). The lowest-index tie-break (§4.4) keeps the trace deterministic on the degenerate problems we test but is not proven to defuse every adversarial tie-heavy or very large instance; tracing through these declined cases is left to future work.

One thread runs throughout. The critical-line iteration is a *parametric active-set homotopy*, the same family as the LARS/LASSO path of statistical learning: a strictly convex quadratic whose linear term moves with a scalar parameter, traced from one breakpoint to the next while an active set is maintained. The portfolio frontier stays in the foreground, but the identical path-tracing engine, with the covariance replaced by a Gram matrix, traces the (constrained) LASSO regularisation path; §10 makes the correspondence literal and runs both through one implementation. We flag it here so the machinery below reads as a general path-follower, not a portfolio-only tool.

The accompanying `cvxcla` package[1] provides the open implementation. Its core is the class `CLA` in `src/cvxcla/cla.py`; it is supported by `first.py` (the starting portfolio), `operators.py` (the covariance backend), and `types.py` (the turning-point and frontier data structures). The notation below follows the code: μ is the vector of expected returns, Σ the covariance, A, b the linear equality constraints, and ℓ, u the box bounds on the weights.

2 Problem formulation

2.1 The parametric quadratic program

Let $w \in \mathbb{R}^n$ be the vector of portfolio weights, $\mu \in \mathbb{R}^n$ the expected returns, $\Sigma \in \mathbb{R}^{n \times n}$ the (symmetric, positive semidefinite) covariance matrix, $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$ the equality constraints, $G \in \mathbb{R}^{p \times n}$ and $h \in \mathbb{R}^p$ the inequality constraints, and $\ell, u \in \mathbb{R}^n$ the lower and upper bounds. The implementation traces the frontier through the *return-parametrised* quadratic program

$$\begin{aligned} \min_{w \in \mathbb{R}^n} \quad & \tfrac{1}{2}\, w^{\mathsf{T}} \Sigma\, w \;-\; \lambda\, \mu^{\mathsf{T}} w \\ \text{subject to} \quad & Aw = b, \\ & Gw \leq h, \\ & \ell \leq w \leq u, \end{aligned} \tag{1}$$

[1] github.com/cvxgrp/cvxcla

where $\lambda \geq 0$ is a scalar that weights expected return against variance. The canonical instance has a single equality constraint, the budget constraint $\mathbf{1}^{\mathsf{T}} w = 1$ (so $A = \mathbf{1}^{\mathsf{T}}$ and $b = 1$), and no inequality rows, but (1) and the implementation admit any m linear equalities and any p linear inequalities: the budget is the single all-ones row, a general A (weighted, or $m > 1$ rows) is handled by a linear-programming first turning point (§3), and a general G (a group- or sector-exposure cap, say) is handled by the bordered turning-point loop of §10. A $\geq$ constraint is expressed by negating its row, so $Gw \leq h$ loses no generality. The box bounds $\ell \leq w \leq u$ are themselves $2n$ inequalities, but per-variable ones, and the method treats them separately and more cheaply (§4.3); $Gw \leq h$ is for the rows that couple several assets at once.

As λ decreases from $+\infty$ to 0, the optimiser of (1) moves from the maximum expected-return portfolio (subject to the constraints) to the global minimum-variance portfolio. The locus of these optimisers is exactly the efficient frontier.

2.2 Optimality conditions

At a fixed λ the program (1) is convex, so the Karush–Kuhn–Tucker (KKT) conditions are necessary and sufficient. Partition the assets into a *free* set $\mathcal{F}$ (weights strictly inside their bounds) and a *blocked* set $\mathcal{B} = \mathcal{F}^c$ (weights pinned at a bound, $w_i \in \{\ell_i, u_i\}$), and let $\mathcal{S} \subseteq \{1, \ldots, p\}$ index the *active* inequality rows, those held at equality $G_{\mathcal{S}} w = h_{\mathcal{S}}$ (the remaining rows are slack and carry a zero multiplier by complementarity). For the free assets the stationarity condition holds with no bound multiplier:

$$\Sigma_{\mathcal{F}\mathcal{F}} w_{\mathcal{F}} + \Sigma_{\mathcal{F}\mathcal{B}} w_{\mathcal{B}} + A_{\mathcal{F}}^{\mathsf{T}} \nu + G_{\mathcal{S}\mathcal{F}}^{\mathsf{T}} \eta \;=\; \lambda \mu_{\mathcal{F}}, \qquad A_{\mathcal{F}} w_{\mathcal{F}} + A_{\mathcal{B}} w_{\mathcal{B}} \;=\; b, \qquad G_{\mathcal{S}\mathcal{F}} w_{\mathcal{F}} + G_{\mathcal{S}\mathcal{B}} w_{\mathcal{B}} \;=\; h_{\mathcal{S}}, \tag{2}$$

where $\nu \in \mathbb{R}^m$ is the multiplier of the equality constraints, $\eta \in \mathbb{R}^{|\mathcal{S}|}$ the (nonnegative) multiplier of the active inequality rows, and the subscripts select the free/blocked rows and columns. The blocked weights $w_{\mathcal{B}}$ are known (they sit on ℓ or u), so (2) is a linear system in the unknowns $(w_{\mathcal{F}}, \nu, \eta)$. An active inequality row enters this system exactly as an equality row does: stacking $C = [\, A;\, G_{\mathcal{S}} \,]$ recovers the equality-only form with A replaced by C, which is precisely what lets the same reduced solve serve both (§10). With no active inequality rows ($\mathcal{S} = \varnothing$) the system collapses to the equality-only KKT.

2.3 Piecewise-linearity: the affine segment

The crucial structural fact is that, *while the free/blocked partition is fixed*, the right-hand side of (2) is affine in λ and the system matrix is constant. Hence the solution is affine in λ:

$$w(\lambda) \;=\; \alpha \;+\; \lambda\, \beta, \tag{3}$$

where $\alpha, \beta \in \mathbb{R}^n$ are constant on the segment (the blocked entries of β are zero; the blocked entries of α equal the active bound values). The implementation calls these vectors `r_alpha` and `r_beta`. Equation (3) is the *critical line*: a straight line in weight space valid for an interval of λ. A *turning point* is a value of λ at which the partition $(\mathcal{F}, \mathcal{B})$ must change, i.e. an endpoint of such an interval.

3 The first turning point

The walk starts at $\lambda = +\infty$, where variance is irrelevant and (1) degenerates into "earn the most expected return that the bounds and budget allow." This portfolio is computed in closed form by `init_algo` (`src/cvxcla/first.py`):

1. Initialise every weight at its lower bound, $w \leftarrow \ell$.

2. Sort the assets by descending expected return.

3. Walk down that order, raising each asset's weight from ℓ_i towards u_i, accumulating capital, until the budget $\sum_i w_i = 1$ is reached (within a floating-point tolerance). The last asset touched is only *partially* filled (it absorbs the residual $1 - \sum_i w_i$) and is the single *free* asset of the first turning point. All others are blocked, either at their lower bound (not yet reached) or upper bound (fully filled).

The result is "the smallest subset of assets with highest return whose upper bounds sum to at least one," which is the maximum-return vertex of the feasible polytope. It is returned as a `TurningPoint` with $\lambda = \infty$, a weight vector, and a boolean `free` mask. The tolerance $1 - 10^{-12}$ on the budget test matters: the increment $1 - \sum_i w_i$ can only reach 1 up to rounding, and without the slack the loop would skip past the genuinely interior asset and mark the next (zero-weight) asset as free. The same greedy fill serves any single all-ones budget row $\mathbf{1}^\mathsf{T} w = b$ (it fills to b rather than to 1), so a leveraged total ($b > 1$) or a dollar-neutral book ($b = 0$, with short positions admitted by a negative ℓ) start from the same construction.

Marking that last asset *free* even when it lands exactly on a bound is deliberate: it guarantees the first turning point has at least one free asset, so the reduced KKT system is non-singular from the outset. The otherwise thorough step-by-step review of Markowitz et al. (2021) does not address this case, in which every asset of the starting portfolio sits on a bound and the reduced solve would degenerate.

For a general equality system $Aw = b$ (weighted coefficients, or $m > 1$ rows such as a budget together with sector- or factor-neutrality) the greedy fill no longer applies, and the maximum-return vertex is instead the solution of the linear program $\max_w \mu^\mathsf{T} w$ subject to $Aw = b$ and $\ell \le w \le u$. `cvxcla` solves it with the HiGHS simplex through `scipy.optimize.linprog`, which returns a vertex; the free set is read off as the assets strictly inside their bounds. When the maximum-return face is degenerate (several vertices attain the same $\mu^\mathsf{T} w$) the simplex returns one of them, fixing only the $\lambda = \infty$ starting vertex: for a positive-definite Σ the optimiser of (1) is unique at every $\lambda > 0$, so the traced frontier below the endpoint does not depend on which maximum-return vertex is chosen. Only the first turning point uses the linear program; the turning-point loop that follows is unchanged (§5), since it is already written for a general A (§8).

4 The turning-point iteration

From the first turning point the algorithm repeatedly: (i) solves the reduced KKT system for the current partition to obtain the affine segment (3); (ii) finds the largest λ below the current one at which an *event* occurs; (iii) updates the partition by the single asset responsible for that event; and (iv) records the new turning point. It stops when λ reaches 0.

4.1 Solving the reduced KKT system by block elimination

For the free partition $\mathcal{F}$ the system (2) is the saddle-point (KKT) system

$$\begin{bmatrix} \Sigma_{\mathcal{F}\mathcal{F}} & A_{\mathcal{F}}^\mathsf{T} \\ A_{\mathcal{F}} & 0 \end{bmatrix} \begin{bmatrix} w_{\mathcal{F}} \\ \nu \end{bmatrix} = \begin{bmatrix} r_1 \\ r_2 \end{bmatrix}. \tag{4}$$

Because the right-hand side splits into a constant part and a λ-part, the implementation solves (4) for *two* right-hand sides simultaneously (the "α system" and the "β system"):

$$r_1^\alpha = -\Sigma_{\mathcal{F}\mathcal{B}}\, w_{\mathcal{B}}, \quad r_2^\alpha = b - A_{\mathcal{B}}\, w_{\mathcal{B}}; \qquad r_1^\beta = \mu_{\mathcal{F}}, \quad r_2^\beta = 0.$$

Rather than assembling and factoring the full saddle matrix, the code eliminates the free block first (*block elimination* / Schur complement). With

$$Y = \Sigma_{\mathcal{F}\mathcal{F}}^{-1} A_{\mathcal{F}}^{\mathsf{T}}, \qquad z^\alpha = \Sigma_{\mathcal{F}\mathcal{F}}^{-1} r_1^\alpha, \qquad z^\beta = \Sigma_{\mathcal{F}\mathcal{F}}^{-1} r_1^\beta,$$

obtained from a *single* multi-column solve against $\Sigma_{\mathcal{F}\mathcal{F}}$, the equality multipliers come from the $m \times m$ Schur complement $S = A_{\mathcal{F}}\, \Sigma_{\mathcal{F}\mathcal{F}}^{-1} A_{\mathcal{F}}^{\mathsf{T}} = A_{\mathcal{F}} Y$:

$$\nu^\alpha = S^{-1}(A_{\mathcal{F}} z^\alpha - r_2^\alpha)\,, \qquad \nu^\beta = S^{-1}\left(A_{\mathcal{F}} z^\beta\right). \tag{5}$$

Back-substitution then gives the free part of the affine segment,

$$\alpha_{\mathcal{F}} = z^\alpha - Y\nu^\alpha, \qquad \beta_{\mathcal{F}} = z^\beta - Y\nu^\beta, \tag{6}$$

while the blocked entries are fixed: $\alpha_{\mathcal{B}} = w_{\mathcal{B}}$ and $\beta_{\mathcal{B}} = 0$. The dominant cost is the multi-right-hand-side solve against the free block $\Sigma_{\mathcal{F}\mathcal{F}}$; the Schur solve (5) is only $m \times m$, and for the standard budget constraint $m = 1$, so it is a scalar division. Bisecting the weights into a free and a blocked part and solving only for the free block keeps every step a small dense solve of size $|\mathcal{F}|$, in contrast to assembling and factoring one large (and, in the mean–semivariance setting, sparse) KKT system over all variables, as the step-by-step construction of Markowitz et al. (2021) does.

4.2 Reduced gradients for blocked assets

To decide when a blocked asset should re-enter the free set we need the reduced gradient of the Lagrangian at the blocked coordinates. The implementation forms two vectors, again split into a constant and a λ-linear part:

$$\gamma = \Sigma\,\alpha + A^{\mathsf{T}}\nu^\alpha, \tag{7}$$

$$\delta = \Sigma\,\beta + A^{\mathsf{T}}\nu^\beta - \mu. \tag{8}$$

For a blocked asset i the quantity $\gamma_i + \lambda\,\delta_i$ is (proportional to) its bound multiplier along the segment. While that multiplier keeps the correct sign the asset is correctly blocked; the λ at which it would change sign is exactly the value at which the asset wants to become free. (The covariance products $\Sigma\alpha$ and $\Sigma\beta$ are applied through the covariance operator of §5, never as an explicit dense multiply when a structured backend is in use.)

4.3 Event detection

Walking down from the current λ, the segment (3) stays optimal until an *event* changes the active set. Events come in two families. The first acts on the *variables* through the box bounds (the four types below, two per direction); the second, present only when $Gw \le h$ has rows, acts on the *constraints* (an inequality row activating or releasing). We describe the box events first:

A free asset reaches a bound (free → blocked). A free weight evolves as $w_i(\lambda) = \alpha_i + \lambda\beta_i$. It meets a bound at

$$\lambda_i \;=\; \begin{cases} \dfrac{u_i - \alpha_i}{\beta_i}, & \beta_i < 0 \quad \text{(heads to the } \textit{upper} \text{ bound)}, \\[2ex] \dfrac{\ell_i - \alpha_i}{\beta_i}, & \beta_i > 0 \quad \text{(heads to the } \textit{lower} \text{ bound)}. \end{cases} \tag{9}$$

A blocked asset's multiplier changes sign (blocked → free). A blocked asset wants to become free at

$$\lambda_i \;=\; -\frac{\gamma_i}{\delta_i}, \tag{10}$$

admissible only when the asset is at its *upper* bound with $\delta_i < 0$, or at its *lower* bound with $\delta_i > 0$.

An inequality row activates or releases (the row analogue). The same two moves occur for the rows of $Gw \le h$, now on *constraints* rather than variables. An inactive row j has slack $s_j(\lambda) = g_j^\mathsf{T} w(\lambda) - h_j = (g_j^\mathsf{T}\alpha - h_j) + \lambda\, g_j^\mathsf{T}\beta$, affine in λ; it becomes active (binding) at the λ where the slack reaches zero from the feasible side. An active row carries a nonnegative multiplier $\eta_j(\lambda) = \eta_j^\alpha + \lambda\,\eta_j^\beta$, also affine in λ; it releases at the λ where that multiplier reaches zero. These are exactly the ratios (9)–(10) with the slack playing the role of a free weight and the row multiplier that of a bound multiplier; §10 gives the construction and the sign conventions.

These candidate ratios are assembled into a matrix $L \in \mathbb{R}^{(n+p)\times 4}$: the first n rows hold the box events of a variable (across the four columns), and the trailing p rows hold the activate/release events of an inequality row, with entries set to $-\infty$ where the event cannot occur. (With no inequality rows, $p = 0$ and L is the $n \times 4$ matrix of the box-and-budget problem.) Two numerical guards apply:

- **Slope floor.** Slopes β_i and derivatives δ_i are tested against $\sqrt{\varepsilon_{\text{mach}}}$, not the user tolerance. A free weight with a *tiny but nonzero* slope still crosses a bound given a long enough λ range, so filtering at the coarser tolerance would let weights drift out of bounds; only slopes at floating-point noise level (below $\sqrt{\varepsilon_{\text{mach}}}$) are discarded, since the enormous ratios they would produce merely amplify rounding error. The exact level is not delicate: genuine slopes sit orders of magnitude above $\sqrt{\varepsilon_{\text{mach}}}$ and round-off noise orders below, so no slope lands near the threshold and the trace is insensitive to its precise value, exactly as the wide regime gap leaves the conditioning guard of §9.6 uncritical.
- **λ window.** The segment is valid only for λ at or below the current value, because the frontier is traced with non-increasing λ. Candidate ratios above the current λ (by more than the tolerance) are spurious crossings outside the segment and are set to $-\infty$.

The next turning point is at $\lambda^\star = \max_{i,t} L_{it}$. If $\lambda^\star < 0$ the frontier is complete.

4.4 Anti-cycling on degenerate problems

On degenerate problems (tied expected returns, duplicated assets, or several constraints active at once) multiple events can occur at the same ratio $\lambda^\star$. A naive "pick any maximiser" rule can cycle (the same partitions recurring without progress). The implementation uses a **Bland-style rule**: among all events within the tolerance of $\lambda^\star$ it selects the one with the lowest asset index, and within an asset the lowest event-type column. This makes the choice deterministic and resolves degeneracies one asset per iteration.

Termination follows by the standard anti-cycling argument. There are only finitely many active-set states (at most 2^{n+p}: the free/blocked split of the n variables and the active subset of the p inequality rows), and each state fixes the affine segment (3) and hence the interval of λ over which it is optimal. The parameter λ is non-increasing along the walk, so the only way to fail to terminate is to revisit a partition at the same $\lambda^\star$: a cycle. Bland's lowest-index selection rule rules this out exactly as it does for the simplex method: among the events tied at $\lambda^\star$ the deterministic lowest-index choice cannot generate a cyclic sequence of partitions. With cycles excluded and the partition set finite, the walk visits each partition at most once and terminates in finitely many steps.

The asset and event type selected determine the partition update: event types "free $\to$ blocked" clear the asset's `free` flag, while "blocked $\to$ free" set it. Exactly one asset changes status. The new weight vector $w(\lambda^\star) = \alpha + \lambda^\star \beta$ is stored as the next turning point.

4.5 Termination and the minimum-variance endpoint

The loop runs while $\lambda > 0$. When no admissible event has a positive ratio the walk has reached the global minimum-variance portfolio; the algorithm appends a final turning point at $\lambda = 0$ with weights α (the constant part of the current segment). Termination is guaranteed in exact arithmetic by the anti-cycling argument of §4.4; the hard iteration cap of $100(n + p + 1)$ is a *numerical* safety net rather than the guarantee itself. A correct trace empirically runs $O(n)$ times, so far exceeding this cap signals a floating-point failure (e.g. a tolerance-induced cycle), which is raised loudly rather than allowed to hang. Every stored turning point is validated to satisfy the bounds, the equality constraints $Aw = b$, and the inequality constraints $Gw \leq h$ before being accepted.

4.6 The complete procedure

Algorithm 1 collects the steps.

Algorithm 1. Critical Line Algorithm (as implemented in `cvxcla`).

Input: mean μ, covariance Σ, bounds ℓ, u, equality constraints A, b, inequality constraints G, h, tolerance τ.

1. Compute the maximum-return vertex $w^{(0)}$ (§3): the greedy `init_algo` when A is the all-ones budget and there are no inequality rows, otherwise the linear program `first_vertex_lp`. Record its free set and its active inequality rows $\mathcal{S}$, and append it with $\lambda \leftarrow \infty$.
2. **While** $\lambda > 0$:
   (a) Read the free/blocked partition $(\mathcal{F}, \mathcal{B})$ and the active inequality rows $\mathcal{S}$ of the last turning point; classify $\mathcal{B}$ into *at-upper* / *at-lower* and fix $w_{\mathcal{B}}$ to the active bounds. Stack the active constraints into $C_{\mathcal{F}} = [\, A_{\mathcal{F}}; G_{\mathcal{SF}} \,]$ (just $A_{\mathcal{F}}$ when $\mathcal{S} = \varnothing$).
   (b) Solve the free block $\Sigma_{\mathcal{FF}}$ for $Y = \Sigma_{\mathcal{FF}}^{-1} C_{\mathcal{F}}^{\mathsf{T}}$ and z^{α}, z^{β} (multi-RHS, §4.1).
   (c) Form the Schur complement $S = C_{\mathcal{F}} Y$ and solve (5) for the equality and active-inequality multipliers $(\nu^{\alpha}, \eta^{\alpha})$ and $(\nu^{\beta}, \eta^{\beta})$; back-substitute (6) to get α, β.
   (d) Compute the reduced gradients γ, δ via (7)–(8).
   (e) Assemble the $(n+p) \times 4$ candidate-ratio matrix L (§4.3): the box events (9)–(10) and the inequality-row activate/release events; apply the slope floor and the λ window. Set $\lambda^{\star} \leftarrow \max L$; **if** $\lambda^{\star} < 0$ **break**.
   (f) Pick the event by the Bland-style rule (§4.4) and flip the status of the single coordinate responsible: the `free` flag of a variable, or the active flag of an inequality row. Set $\lambda \leftarrow \lambda^{\star}$ and append the turning point $w \leftarrow \alpha + \lambda^{\star} \beta$.
3. Append the final turning point $w \leftarrow \alpha$ at $\lambda = 0$ (the minimum-variance portfolio) and **return** the list of turning points.

5 The covariance operator abstraction

The turning-point loop touches the covariance matrix through only three operations: a full matrix–vector product Σx (`matvec`); a solve against the free block, $\Sigma_{\mathcal{FF}}^{-1} R$ for a multi-column right-hand side R (`solve_free`); and a free-to-blocked cross product $\Sigma_{\mathcal{FB}} x_{\mathcal{B}}$ (`cross`). The package captures exactly this contract in the `QuadraticForm` protocol (`src/cvxcla/operators.py`; `CovarianceOperator` remains as an alias), so the algorithm never assumes an explicit matrix. The name is deliberately not tied to covariance: the same contract is the Hessian of any quadratic objective, a point we return to in §10. Four backends implement it.

5.1 Dense covariance

`DenseCovariance` wraps an explicit symmetric $n \times n$ matrix and implements the three operations with plain `numpy` calls: `matvec` is Σx, `solve_free` is `np.linalg.solve` on the principal submatrix $\Sigma_{\mathcal{FF}}$, and `cross` is the off-diagonal block product. This reproduces exactly the behaviour of the classical CLA on a raw matrix and is the reference backend.

5.2 Incremental dense covariance

`IncrementalDenseCovariance` wraps the same explicit matrix and returns identical results, but computes `solve_free` differently. The reference backend factorises $\Sigma_{\mathcal{F}\mathcal{F}}$ from scratch at every turning point, an $O(n_{\mathcal{F}}^3)$ cost. Because the CLA changes the free set by exactly one asset between consecutive turning points (§4.4), this backend instead *maintains* the explicit inverse $\Sigma_{\mathcal{F}\mathcal{F}}^{-1}$ and updates it in place as the free set changes. When an asset enters the free set, a rank-one bordered update appends its row and column through the Schur scalar $s = \Sigma_{aa} - c^{\mathsf{T}}\Sigma_{\mathcal{F}\mathcal{F}}^{-1}c$ (with $c = \Sigma_{\mathcal{F}a}$ the new cross-column); when an asset leaves, the symmetric deletion formula $\Sigma_{\mathcal{F}'\mathcal{F}'}^{-1} = B_{\setminus p} - B_{\cdot p}B_{p\cdot}/B_{pp}$ drops its row and column from the current inverse B. Each update costs $O(n_{\mathcal{F}}^2)$ rather than $O(n_{\mathcal{F}}^3)$, so the linear-algebra strategy matches the incremental-inverse baseline of §9.3: on a dense problem of a few hundred assets it runs roughly twice as fast per solve, and traces the identical frontier to machine precision.

The trade is numerical. A maintained inverse accumulates floating-point error over the $O(n)$ rank-one updates of a trace, whereas the fresh solve is clean at every step; a non-positive or non-finite Schur pivot, or any free-set change that is not a single-asset flip, triggers a full refactorisation as a guard. For well-conditioned, positive-definite problems the two backends agree to solver precision, but on ill-conditioned or near-degenerate inputs the plain `DenseCovariance`, or the factor backend below, is the safer choice. This backend is therefore offered as an opt-in fast path, not the default, and the win is dense-only: the factor backend never forms $\Sigma_{\mathcal{F}\mathcal{F}}$ at all, so there is no inverse to maintain.

5.3 Factor (diagonal-plus-low-rank) covariance

`FactorCovariance` represents a structured risk model

$$\Sigma \;=\; \operatorname{diag}(d) \;+\; U\,\Delta\,U^{\mathsf{T}}, \qquad d \in \mathbb{R}^{n}_{>0},\; U \in \mathbb{R}^{n\times k},\; \Delta \in \mathbb{R}^{k\times k}, \tag{11}$$

the form taken by factor models (Sharpe, 1963) and by random-matrix-theory-cleaned covariances (constant-residual-eigenvalue / eigenvalue clipping). The free-block solve never forms $\Sigma_{\mathcal{F}\mathcal{F}}$; instead it uses the Woodbury identity

$$\Sigma_{\mathcal{F}\mathcal{F}}^{-1} \;=\; D_{\mathcal{F}}^{-1} \;-\; D_{\mathcal{F}}^{-1}U_{\mathcal{F}}\,W^{-1}\,U_{\mathcal{F}}^{\mathsf{T}}D_{\mathcal{F}}^{-1}, \qquad W \;=\; \Delta^{-1} + U_{\mathcal{F}}^{\mathsf{T}}D_{\mathcal{F}}^{-1}U_{\mathcal{F}} \;\in\; \mathbb{R}^{k\times k}, \tag{12}$$

so a solve costs $O(n_{\mathcal{F}}k^2 + k^3)$ rather than $O(n_{\mathcal{F}}^3)$, and no $n \times n$ matrix is ever allocated: memory and per-solve cost scale as $O(nk)$ instead of $O(n^2)$. The cross product (11) contributes only through its low-rank part, since the diagonal has no off-diagonal block: $\Sigma_{\mathcal{F}\mathcal{B}}\,x_{\mathcal{B}} = U_{\mathcal{F}}\,\Delta\,(U_{\mathcal{B}}^{\mathsf{T}}x_{\mathcal{B}})$. Because the CLA accesses the covariance solely through the protocol, switching from a dense matrix to a factor model requires no change to the turning-point loop.

5.4 Data-matrix (Gram) covariance

A sample covariance is itself a Gram matrix. If $X_c \in \mathbb{R}^{T\times n}$ is the column-centred matrix of T return observations, then

$$\Sigma \;=\; \frac{1}{T-1}\,X_c^{\mathsf{T}}X_c, \tag{13}$$

and `GramCovariance` implements the protocol straight from X_c, never forming the $n \times n$ matrix: `matvec` is the pair of thin products $X_c^{\mathsf{T}}(X_c x)/(T-1)$, `cross` restricts those products to the free and blocked columns, and `solve_free` works on $X_{c\mathcal{F}}$ alone. Memory is $O(Tn)$ rather than $O(n^2)$.

The catch is rank: $X_c^\mathsf{T} X_c$ has rank at most $T-1$. When there are at least as many observations as assets ($T \geq n$, with X_c of full column rank) Σ is positive definite and this backend is the dense reference at lower memory. In the short-sample regime $T < n$ (a covariance estimated from far fewer periods than assets) Σ is singular, and the free block becomes unsolvable once the free set outgrows the data rank: exactly the degeneracy the algorithm detects through `rcond_free`. An optional ridge δI restores positive definiteness, and the free-block solve is then the Woodbury identity (12) carried out in the T-dimensional observation space (cost $O(n_\mathcal{F} T^2 + T^3)$). This is precisely a factor model (11) whose factors are the observations themselves ($U = X_c^\mathsf{T}$, $k = T$).

This is a memory benefit, not an unconditional speed-up. When $T \geq n$ and the dense covariance fits in memory, `DenseCovariance` is faster per turning point: it slices a precomputed $\Sigma_{\mathcal{F}\mathcal{F}}$, whereas the Gram backend re-forms $X_{c\mathcal{F}}^\mathsf{T} X_{c\mathcal{F}}$ at each solve. The same caveat governs the factor backend (§9.4): a structured solve pays only while the rank, or here the observation count T, stays well below n; at full rank the structured route is slower than the plain dense matrix.

5.5 Regularization through the operator

A regularized objective is supported whenever the penalty is *quadratic*, and for the same reason the bare algorithm works: a quadratic penalty changes only the Hessian, so the KKT system stays linear in λ and the path stays piecewise linear. A ridge (Tikhonov) term is the canonical case,

$$\tfrac{1}{2}\, w^\mathsf{T} \Sigma\, w - \lambda\, \mu^\mathsf{T} w + \tfrac{\gamma}{2}\, \|w\|_2^2 \;=\; \tfrac{1}{2}\, w^\mathsf{T} (\Sigma + \gamma I)\, w - \lambda\, \mu^\mathsf{T} w, \qquad \gamma \geq 0, \tag{14}$$

which is exactly the substitution $\Sigma \mapsto \Sigma + \gamma I$, and more generally $\Sigma + \gamma M$ for any positive-semidefinite M (a factor-exposure penalty, a deviation-from-benchmark term, a quadratic turnover penalty $\|w - w_{\text{prev}}\|_2^2$). Because the turning-point loop reaches the covariance only through the operator protocol of §5, the regularized form is just another `QuadraticForm`: the penalty is supplied by the *backend* and the loop is untouched, each step still a single symmetric positive-definite solve. The result is the exact frontier of the regularized objective.

Two backends already realise this. `GramCovariance` takes a ridge $\delta \geq 0$ directly (§5.4), representing $\Sigma = X_c^\mathsf{T} X_c/(T-1) + \delta I$, and `FactorCovariance` (11) is itself a structured ridge, positive definite by the idiosyncratic floor $d > 0$. A quadratic regularizer is therefore not merely permitted but recommended where it is needed: it is precisely the conditioning fix for the rank-deficient free block of §9.6, where the unregularized trace is declined. A small γ, or a factor model, keeps the free block full rank and the trace completes.

The boundary follows from the same quadratic requirement. Shrinkage toward a *nonzero* target w_0, $\frac{\gamma}{2}\|w - w_0\|_2^2$, is still quadratic and compatible in principle, but it adds a λ-independent linear term $-\gamma\, w_0^\mathsf{T} w$ that the present interface, whose linear term is exactly $-\lambda\mu$, does not expose. A nonsmooth ℓ_1 penalty $\tau\|w\|_1$ is not a term in this objective at all: it is piecewise linear, vacuous for the long-only fully-invested book (where $\|w\|_1 = \mathbf{1}^\mathsf{T} w = 1$ is constant), and where it does bite (gross exposure with short positions) it is traced by the LASSO homotopy over τ rather than the risk-aversion sweep over λ, the correspondence of §10; a gross-exposure *constraint* $\|w\|_1 \leq c$ is instead polyhedral, expressible as $Gw \leq h$ under the split $w = w^+ - w^-$. Genuinely nonquadratic penalties (an entropy or log-barrier term) break the linear-in-λ structure and lie outside the method.

6 The frontier object and its properties

The list of turning points is wrapped in a `Frontier` object (`types.py`). Each `FrontierPoint` carries a weight vector and exposes its expected return $\mu^\mathsf{T} w$ and variance $w^\mathsf{T} \Sigma w$. Between adjacent turning points the frontier in *weight* space is the straight segment (3), so the object can

`interpolate` any number of intermediate portfolios by convex combination. Derived quantities (volatility, Sharpe ratio) are computed pointwise. The maximum-Sharpe portfolio is computed in closed form, exploiting the same piecewise-linear weight structure. On an affine weight segment $w(t) = (1-t)\,w_0 + t\,w_1$, $t \in [0,1]$, the expected return is affine and the variance quadratic in t, so the Sharpe ratio $S(t) = (a_0 + a_1 t)/\sqrt{c_0 + c_1 t + c_2 t^2}$ is an affine function over the square root of a quadratic (here $a_0 = \mu^\mathsf{T} w_0$, $a_1 = \mu^\mathsf{T}(w_1 - w_0)$, and c_0, c_1, c_2 are the coefficients of $w(t)^\mathsf{T}\Sigma\, w(t)$). Its derivative has a *linear* numerator, the quadratic terms cancelling, so each segment has a single stationary point $t^\star = (a_0 c_1 - 2a_1 c_0)/(a_1 c_1 - 2a_0 c_2)$; the maximiser is the best of $t^\star$ (when it lies in $[0,1]$) and the two segment endpoints, taken over the two segments bracketing the discrete Sharpe maximum. Two segments suffice because the Sharpe ratio is unimodal along the frontier: the efficient frontier is concave and increasing in the $(\sigma, \mu^\mathsf{T} w)$ plane, so the ray slope $\mu^\mathsf{T} w/\sigma$ rises to the tangency portfolio and falls thereafter; its values at the turning points are therefore unimodal, and the turning point of largest discrete Sharpe ratio brackets the continuous maximum. The frontier object therefore returns the maximum-Sharpe portfolio exactly, not to a search tolerance.

Four properties of the traced frontier are worth recording explicitly.

- **Exactness.** The frontier is returned exactly as a finite set of turning points; between them the closed-form segment (3) holds. No discretisation error is introduced.
- **Number of turning points.** Each iteration changes exactly one coordinate's status: a variable entering or leaving the free set, or an inequality row activating or releasing. On non-degenerate problems a coordinate re-enters the free set only finitely often, so the count is empirically $O(n)$, mirroring the typical $O(n)$ breakpoint count of the LARS path (Efron et al., 2004); the worst case is combinatorial (as for parametric active-set and mpQP methods generally (Bemporad et al., 2002)) but is not observed on the problems of §9. The implementation caps iterations at $100(n + p + 1)$ (one factor over the n box bounds and p inequality rows whose activity each turning point fixes) as a numerical safety net.
- **Per-iteration cost.** Dominated by the free-block solve: $O(n_\mathcal{F}^3)$ for the dense backend, or $O(n_\mathcal{F} k^2 + k^3)$ for the factor backend via Woodbury (12). The Schur complement (5) adds only an $(m + |\mathcal{S}|) \times (m + |\mathcal{S}|)$ solve over the equality rows plus the active inequality rows ($m = 1$, $\mathcal{S} = \varnothing$ for the plain budget constraint).
- **Robustness on degenerate data.** The Bland-style tie-break (§4.4) and the $\sqrt{\varepsilon_{\text{mach}}}$ slope floor (§4.3) make the trace deterministic and prevent cycling and out-of-bounds drift on the degenerate problems we test (tied means, capped weights, slow bound crossings), and sub-tolerance round-off at degenerate vertices is projected back onto the feasible set so near-rank-deficient problems still complete with optimal turning points (§9.6). The tie-break is robust on the degenerate problems we test but is not proven to defuse every degeneracy; §9.6 states this boundary precisely.

7 Using the package

The package is installed with `pip install cvxcla` and the whole workflow goes through a single class, `CLA`. The user supplies the mean μ, the covariance Σ (a dense array or any backend of §5), the box bounds ℓ, u, and the constraints: the equality system A, b (always at least the budget), and optionally the inequality system G, h. Constructing the object traces the whole frontier; the turning points and the derived `Frontier` (§6) are then available as attributes. Listing 1 traces a small long-only, fully-invested problem and reads off the maximum-Sharpe portfolio.

Listing 1: Tracing a frontier and reading the maximum-Sharpe portfolio.

```
import numpy as np
from cvxcla import CLA, FactorCovariance

rng = np.random.default_rng(1)
n, k = 6, 2
d = rng.uniform(0.1, 0.5, n) # idiosyncratic variances
U = rng.standard_normal((n, k)) # factor loadings
Delta = np.diag(rng.uniform(0.5, 2.0, k))
Sigma = np.diag(d) + U @ Delta @ U.T # a dense 2-factor covariance
mean = rng.standard_normal(n)

cla = CLA(
    mean=mean, covariance=Sigma,
    lower_bounds=np.zeros(n), upper_bounds=np.ones(n),
    a=np.ones((1, n)), b=np.ones(1), # fully invested: sum(w) = 1
)
print("turning points:", len(cla.turning_points))
sr, w = cla.frontier.max_sharpe
print("max Sharpe:", round(sr, 4))
print("weights:", np.round(w, 3))
```

```
turning points: 7
max Sharpe: 2.4037
weights: [0. 0. 0.704 0.296 0. 0. ]
```

The constructor takes the problem in its raw matrix form, which is explicit but verbose for the common cases. The same problem can be assembled through a fluent builder, `CLA.problem(...)`, whose named steps each map onto one constructor argument: `.long_only()` sets the box bounds $0 \le w \le u$, `.budget()` adds the fully-invested row $\sum_i w_i = 1$, and the terminal `.trace()` builds the `CLA` and runs the walk. The builder is pure sugar: it adds no modelling power and accepts only the same polyhedral pieces, so it returns the identical object and frontier (Listing 2). `.bounds(lower, upper)`, `.equality(A, b)`, and `.inequality(G, h)` add the general bounds and constraints discussed below.

Listing 2: The same problem assembled through the fluent builder; `.trace()` returns the identical `CLA`.

```
from cvxcla import CLA

cla = (
    CLA.problem(mean, Sigma) # same mean and covariance as above
       .long_only() # box bounds 0 <= w <= 1
       .budget() # fully invested: sum(w) = 1
       .trace() # build the CLA and trace the frontier
)
print("turning points:", len(cla.turning_points))
print("max Sharpe:", round(cla.frontier.max_sharpe[0], 4))
```

```
turning points: 7
max Sharpe: 2.4037
```

Switching to a structured backend changes only the covariance argument: the turning-point loop is untouched, and because the factor backend represents the *same* Σ it returns the identical frontier (§5), solving through the Woodbury identity instead of forming Σ (Listing 3). A short-sample sample covariance is handled the same way through `GramCovariance`, built straight from the centred return matrix.

Listing 3: The factor backend traces the identical frontier without ever forming the $n \times n$ matrix.

```
factor = FactorCovariance(d=d, u=U, delta=np.diag(Delta))
cla_factor = CLA(
    mean=mean, covariance=factor,
    lower_bounds=np.zeros(n), upper_bounds=np.ones(n),
    a=np.ones((1, n)), b=np.ones(1),
)
w_dense = np.array([t.weights for t in cla.turning_points])
w_factor = np.array([t.weights for t in cla_factor.turning_points])
print("identical frontier:", np.allclose(w_dense, w_factor))
```

```
identical frontier: True
```

General linear constraints are passed the same way, exactly the program (1): a weighted or multi-row A, b for sector or factor neutrality, and G, h for group or sector exposure caps (a $\geq$ floor is the negated row). The builder accumulates rows, so a neutrality block sits alongside the budget while the box bounds remain the cheaper per-variable active set of §4.3. Listing 4 adds a sector-neutrality row to the budget (a two-row A), so the first turning point is the linear program of §3 rather than the greedy fill, and the traced frontier holds the sector exposure at its target throughout.

Listing 4: A multi-row equality A, b: budget plus a sector-neutrality row. Reuses `mean`, `Sigma` from Listing 1.

```
sector = np.arange(n) % 3 # split the universe into 3 sectors
in_sector0 = (sector == 0).astype(float)

cla = (
    CLA.problem(mean, Sigma)
       .long_only() # 0 <= w <= 1
       .budget() # sum(w) = 1
       .equality(in_sector0, 0.30) # hold 30% in sector 0: A now has 2 rows
       .trace()
)
print("equality rows:", cla.a.shape[0])
print("sector-0 weight:", round(float(in_sector0 @ cla.frontier.max_sharpe[1]), 2))
```

```
equality rows: 2
sector-0 weight: 0.3
```

A coupling inequality is added the same way. Listing 5 caps each sector's exposure with a block $Gw \leq h$; the high-return end would overweight a sector, so the cap is *active* at every turning point of this frontier.

Listing 5: An inequality block $Gw \leq h$: a per-sector exposure cap, active along the whole trace.

```
G = np.array([(sector == s).astype(float) for s in range(3)]) # one row per sector
h = np.full(3, 0.45) # each sector <= 45%
```

```
cla = CLA.problem(mean, Sigma).long_only().budget().inequality(G, h).trace()
binding = sum(int(np.any(np.abs(G @ tp.weights - h) <= 1e-7))
              for tp in cla.turning_points)
print("turning points:", len(cla.turning_points), "| binding-cap points:", binding)
```

```
turning points: 6 | binding-cap points: 6
```

Both trace the exact constrained frontier, not an approximation: §9.5 validates each against a per-λ general QP solve, including the regime where the inequality caps bind. Every figure and table in the remainder of this paper is regenerated by a single replication script, `reproduce_paper.py`, which runs each experiment in turn off the committed inputs (no network access) and reports which artefact it produces; the individual scripts it calls are cited at the relevant figures and tables below.

8 Relation to the Bailey–López de Prado implementation

The most widely used open implementation of the CLA is that of Bailey and López de Prado (2013) (the implementation behind PyPortfolioOpt's `CLA` (Martin, 2021), against which we benchmark in §9). That paper is the reference open-source CLA; when published it filled a real gap, as the only prior documented implementation was the Markowitz–Todd VBA-Excel code of Markowitz and Todd (2000). Our implementation in fact shares several of its design choices: the same greedy first turning point (§3), the explicit minimum-variance endpoint at $\lambda = 0$, and locating the maximum-Sharpe portfolio between bracketing turning points (§6, in closed form here). It traces the same turning points as the method described here, since the underlying mathematics is Markowitz's, but differs in four ways that matter in practice.

1. **Covariance representation.** Bailey–López de Prado operate on an *explicit dense covariance matrix*: each turning point forms and inverts the free-block covariance $\Sigma_{\mathcal{F}\mathcal{F}}$ directly (their `getMatrices` / `reduceMatrix` helpers slice the stored matrix). `cvxcla` never requires the matrix: it reaches the covariance only through the operator protocol of §5, so structured backends (the diagonal-plus-low-rank Woodbury solve) plug in unchanged. This is the central difference and the source of the asymptotic speed-up in §9.4.

2. **Linear algebra.** Where the reference algorithm forms an explicit inverse of the free block from scratch at each step (and, in the branch that decides which blocked asset re-enters, recomputes that full inverse once for every candidate), `cvxcla` casts each step as the reduced KKT saddle-point solve of §4.1: a single block-eliminated solve against $\Sigma_{\mathcal{F}\mathcal{F}}$ feeding an $m \times m$ Schur complement, with the event search over all candidates vectorised rather than looped. More fundamentally, Bailey–López de Prado never assemble a KKT system at all: they substitute an explicit $\Sigma_{\mathcal{F}\mathcal{F}}^{-1}$ into closed-form Lagrange expressions for λ, γ, and $w_{\mathcal{F}}$ specialised to the single budget row. The saddle-point formulation is what lets `cvxcla` reach the covariance as a black-box solve (the backend abstraction) and take an arbitrary A (next point) where their closed-form algebra admits only one.

3. **Constraint generality.** The reference implementation targets box bounds plus a single budget (fully-invested) constraint. `cvxcla` admits an arbitrary set of m linear equality constraints $Aw = b$ and p linear inequality constraints $Gw \leq h$; the budget is just the case $A = \mathbf{1}^\mathsf{T}$, $b = 1$, $p = 0$, and the Schur complement scales to $m + |\mathcal{S}| > 1$ active rows at negligible cost. The turning-point loop is general in A throughout, and an active inequality row

is carried as an extra row of that Schur complement (§10); the only constraint-specific step is the maximum-return vertex that seeds it, found by the greedy fill for an all-ones budget and by a linear program (HiGHS, via `scipy`) for a general A or any G (§3). This covers leverage, dollar-neutral long/short books, multi-row neutralities (sector, factor), and group- or sector-exposure caps. One caveat mirrors the degeneracy guarantee of §9.6: the general-A path assumes a well-conditioned first vertex, where the free set has the m assets the equalities require. At a *degenerate* vertex (a basic asset pinned exactly on a box bound, so the free set is too small to span the constraints) the reduced $A_{\mathcal{F}}$ loses row rank and the Schur complement is singular. That vertex is *declined* at the first turning point with an actionable diagnosis rather than mis-solved; tracing through it (rather than declining) is left to future work alongside the tie-heavy hardening of §9.6.

4. **Degeneracy handling.** On tie-heavy or degenerate inputs several events can share the same critical λ. `cvxcla` applies an explicit Bland-style, lowest-index tie-break (§4.4) together with a floating-point slope floor (§4.3) to keep the trace deterministic and bounded; the reference implementation has no comparable anti-cycling rule. It also projects the sub-tolerance round-off that near-degenerate vertices produce back onto the feasible set, so near-rank-deficient covariances trace to completion with optimal turning points; only a genuinely rank-deficient free block is declined (§9.6).

These differences compound into a large runtime gap (§9). It would be tempting to dismiss the gap as "just an implementation detail," but the reference implementation is not slow for lack of `numpy`: it uses `numpy` for its per-step algebra. It is slow because of the structure above: a full free-block inverse rebuilt every step, and rebuilt once per candidate in the asset-entry search. §9.3 pins this down with a controlled baseline.

Relation to large-scale active-set methods. Bailey–López de Prado is the natural baseline because it is the most widely used open CLA, but it is not the only line of work on tracing the frontier efficiently. Perold (1984), Stein et al. (2008) and Hirschberger et al. (2010) develop parametric quadratic-programming and active-set methods aimed squarely at *large* universes, and they too drive the per-step cost well below a dense $O(n_{\mathcal{F}}^3)$ refactorisation. The decisive difference is *where* the speed-up comes from. Those methods accelerate the linear algebra of a covariance that is still treated as an explicit (if large and possibly sparse) matrix (through incremental factorisation updates, exploitation of sparsity, or specialised pivoting) so the gain is tied to a particular matrix representation baked into the solver. `cvxcla` instead leaves the turning-point loop entirely representation-agnostic: the covariance enters only through the three-operation operator protocol of §5, so the same loop runs unchanged on a dense matrix or on a diagonal-plus-low-rank factor model, and the asymptotic advantage of §9.4 is a property of the *backend*, not of the algorithm. The two ideas are complementary rather than competing: an incremental-update solver for the free block could itself be wrapped as a `CovarianceOperator` and dropped in. A direct empirical comparison is left to future work, for a concrete reason: no maintained, installable fast large-scale CLA exists to benchmark against. The closest is the Fortran implementation of Niedermayer and Niedermayer (2010) (with MATLAB and Python bindings), which reports speedups of several orders of magnitude over earlier CLA codes but is effectively unmaintained and not buildable on current toolchains without nontrivial effort; Hirschberger et al. (2010)'s "CIOS" is self-contained Java research code; and a general parametric active-set QP solver such as qpOASES would have to be driven along λ with warm starts rather than tracing the frontier natively. Reviving the Niedermayer code, or a λ-swept qpOASES, is the natural baseline for such a comparison. The reading here is that `cvxcla`'s

contribution is the interface that makes such backends interchangeable, not a claim to the fastest dense large-scale solve.

Relation to portfolio-optimization software. Set against the broader ecosystem of portfolio-optimization packages, the distinction is less speed than *what is computed*. General toolkits, in Python Riskfolio-Lib (Cajas, 2021) and the scikit-learn-based `skfolio` (Nicolini et al., 2025), and in R `fPortfolio` (Würtz et al., 2009) and the disciplined-convex layer `CVXR` (Fu et al., 2020), construct a portfolio by solving a (mean–variance or richer) optimisation at a *single* risk target, typically by handing a quadratic or conic program to a general solver; recovering the frontier then means re-solving on a grid of targets, an approximation that the grid resolution controls. `cvxcla`, like the CLA itself and like PyPortfolioOpt's `CLA` (Martin, 2021), instead returns the *entire* frontier exactly as a finite set of turning points, with no grid. Among the exact-CLA tools, PyPortfolioOpt is the natural head-to-head (§9) but, as noted there, is not a performance-tuned solver. What is genuinely particular to `cvxcla` is orthogonal to all of these: the operator interface and the structured factor and Gram backends (§5), which run the same exact-frontier trace directly on a covariance representation the general toolkits do not exploit.

9 Numerical experiment

To illustrate the algorithm we first trace the entire efficient frontier of a small problem (small enough that its piecewise-linear corner structure is plainly visible) and then measure how the runtime grows with the number of assets. Both experiments are reproducible from fixed random seeds: `experiments/frontier_20x50.py` and `experiments/runtime_scaling.py`.

Benchmark environment. The runtime study below (the scaling sweep of §9.2 and the rank sweep of §9.4, i.e. Tables 1–2 and Figures 2–3) was run on a single machine: an Apple M4 Pro (14 cores, 48 GB RAM) running macOS 15.6.1, with Python 3.12.7 and NumPy 2.4.6 linked against the Apple Accelerate BLAS, and PyPortfolioOpt 1.6.0. Absolute times are therefore machine-specific and BLAS-dependent; the empirical scaling exponents and the relative speedups between implementations, which are the quantities we draw conclusions from, are far more portable than the raw milliseconds. Each reported timing is the median of three repetitions on identical inputs (the bands in the figures span their range). Beyond the one-off experiment scripts cited above, the relative backend performance is tracked continuously by a committed `pytest-benchmark` suite (`tests/benchmarks/`) run in CI, so the comparisons here are reproducible independently of this machine.

9.1 Illustration: a 20-asset frontier

Setup. We simulate $T = 50$ daily returns for $N = 20$ assets from a $K = 5$ factor model. As the risk model we use the factor covariance itself, $\Sigma = \operatorname{diag}(d) + U\,\Delta\,U^{\mathsf{T}}$ (standard practice, a structured risk model rather than the noisy sample covariance), and we estimate the expected returns $\mu \in \mathbb{R}^{20}$ by the sample mean over the 50 days. We trace the long-only, fully-invested frontier, i.e. (1) with the budget constraint $\mathbf{1}^{\mathsf{T}} w = 1$ and box bounds $0 \le w \le 1$.

Results. The algorithm returns the *entire* frontier as a sequence of **21 turning points** (Figure 1); between consecutive points the frontier is the exact affine segment (3), so these 21 corner portfolios describe the whole efficient set with no discretisation. The piecewise-linear corners are clearly visible

towards the low-variance end of the curve. Tracing this small problem takes on the order of a millisecond. Because the dense matrix and the structured `FactorCovariance` (Woodbury) backend represent the *same* Σ, the two backends return the identical 21-point frontier: the Woodbury identity is exact, so it changes only the cost of each solve, never the result; its runtime advantage as the universe grows is the subject of §9.2. (As an aside, replacing Σ by a low-rank *approximation* of the full sample covariance (its leading eigenpairs plus a diagonal residual) yields a near-identical frontier differing by at most one corner, which is why factor risk models are a practical substitute for the raw sample covariance.)

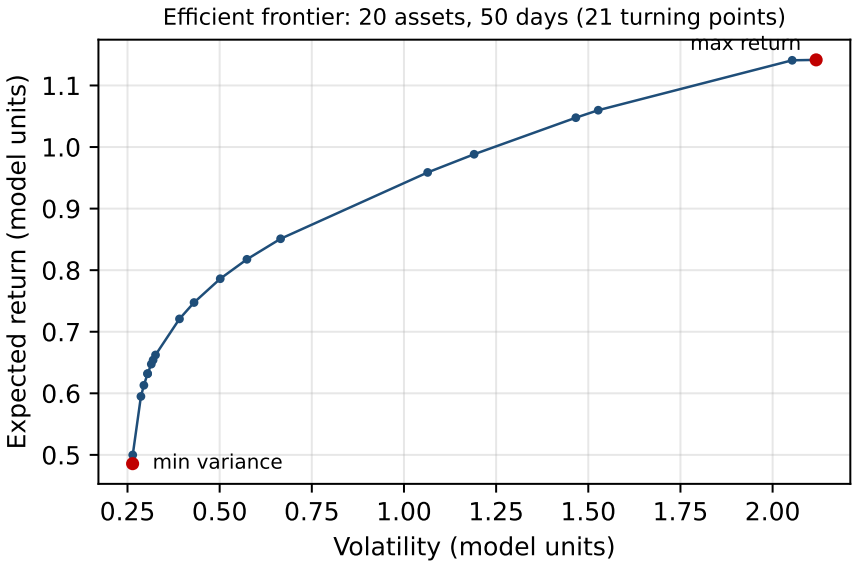

Figure 1: The efficient frontier of the 20-asset, 50-day problem in expected-return / volatility space, computed exactly by the Critical Line Algorithm. Each marker is one of the 21 turning points; the endpoints are the maximum-return portfolio ($\lambda = \infty$) and the global minimum-variance portfolio ($\lambda = 0$). Axes are in the dimensionless "model units" of the simulated factor model (per-period return and volatility of the synthetic process); unlike the annualised S&P 500 frontier of Figure 4, they carry no calendar scale and are meaningful only relative to one another.

9.2 Runtime versus problem size

We next trace the full frontier for $n \in \{20, 40, 80, 160, 320, 640\}$ assets, each from a ground-truth $K = 10$ factor model so that the dense and factor backends again solve the identical Σ and return the same frontier. For reference we also time the `CLA` of PyPortfolioOpt (Martin, 2021), a widely-used implementation of the critical-line algorithm of Bailey and López de Prado (2013), on the same dense problem. We time it because it is the most familiar open CLA, not because it is a tuned competitor; the gap below therefore measures the cost of how its per-step work is organised (§9.3), not `cvxcla` against a performance-optimised solver. The number of turning points tracks n closely ($21, 42, 81, 159, 322, 641$ respectively, and PyPortfolioOpt finds nearly the same set: $19, 42, 81, 158, 322, 641$), confirming the $O(n)$ count of §9. The small count differences are not noise but a completeness gap: PyPortfolioOpt's turning points are a strict *subset* of `cvxcla`'s (every PyPortfolioOpt corner is matched), and `cvxcla` finds one or two *additional* genuine corners that PyPortfolioOpt omits (the omitted weights differ from PyPortfolioOpt's nearest reported corner by

up to $\sim 2 \times 10^{-2}$, well above round-off). Since `cvxcla`'s frontier is independently validated exact (§9.5), these are corners PyPortfolioOpt misses, a (minor) correctness difference, not merely a speed one. The two implementations also agree numerically: their minimum-variance portfolios (the $\lambda = 0$ endpoint, which is unique for a positive-definite Σ) match to machine precision, differing by at most $\sim 10^{-16}$ in any weight across these problems.

Figure 2 plots the median trace time against n on log–log axes. A least-squares fit over the largest four sizes gives an empirical exponent of ≈ 2.2 for the `cvxcla` dense backend and ≈ 1.3 for the factor backend: the dense per-iteration solve scales like $O(n_{\mathcal{F}}^3)$ over $O(n)$ iterations, while the Woodbury solve with fixed K stays close to linear in n. The dense and factor backends are comparable for small universes (the Woodbury correction carries a fixed $K \times K$ overhead), but the factor backend pulls away as n grows: from rough parity at $n = 40$ to a $\approx 7.7\times$ speedup at $n = 640$ (Table 1). This $\approx 7.7\times$ is the genuine *algorithmic* gain: both backends are vectorised `numpy` solving the identical Σ, so the only difference is the asymptotic Woodbury advantage of the structured solve over the dense one.

For external context we also time PyPortfolioOpt, which scales far more steeply ($\approx n^{3.4}$): at $n = 640$ it takes about 255 seconds, so `cvxcla`'s dense backend is roughly $322\times$ faster and the factor backend roughly $2460\times$ faster. This gap is not a `numpy`-versus-Python artefact (PyPortfolioOpt uses `numpy` for its linear algebra) but a consequence of how it organises the work: it rebuilds a full free-block inverse at every step, and once per candidate in the asset-entry search (§8). §9.3 separates this structural cost from the genuinely *algorithmic* advantage, which is the structured factor backend.

n	Turning points	PyPortfolioOpt	Inverse baseline	Dense	Factor
20	21	7.1	0.7	1.2	1.6
40	42	39.9	2.0	3.5	3.5
80	81	207.9	4.8	7.7	6.9
160	159	1,470	12.7	26.1	14.8
320	322	16,632	50.1	122.7	36.5
640	641	254,726	328.6	792.0	103.6

Table 1: Frontier trace time (milliseconds) versus number of assets n ($K = 10$ factors). The "Dense" and "Factor" columns are the two `cvxcla` backends. All four implementations are timed with the *same* protocol (the median of 3 repetitions on the same machine and inputs) and return the same $O(n)$-point frontier. "Inverse baseline" is the incremental-inverse CLA of §9.3 (`experiments/inverse_cla.py`): it shares `cvxcla`'s vectorised event logic but maintains $\Sigma_{\mathcal{F}\mathcal{F}}^{-1}$ through rank-one updates instead of re-solving each step, so the gap between it and the dense backend isolates the linear-algebra strategy. Empirically the times scale like $\approx n^{3.4}$ (PyPortfolioOpt), $\approx n^{2.0}$ (inverse baseline), $\approx n^{2.2}$ (`cvxcla` dense) and $\approx n^{1.3}$ (`cvxcla` factor).

9.3 What the runtime gap is, and is not

The $\approx 322\times$ gap between `cvxcla`'s dense backend and PyPortfolioOpt at $n = 640$ invites a lazy explanation ("vectorised `numpy` versus slow Python") that is wrong. PyPortfolioOpt already uses `numpy` (`np.linalg.inv`, `np.dot`) for its per-step linear algebra. It is slow because of *how* it organises the work: it rebuilds a full inverse of the free block from scratch at every step, and in the branch that decides which blocked asset should re-enter it recomputes that full $O(n_{\mathcal{F}}^3)$ inverse once for *every* candidate blocked asset, of order n dense inverses per turning point (in the reference code of

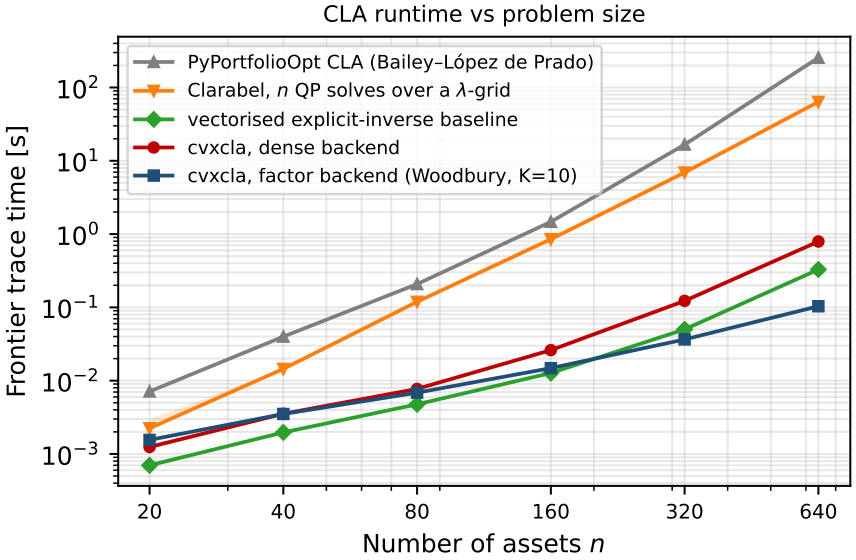

Figure 2: Frontier trace time as a function of the number of assets n (log–log). PyPortfolioOpt (Martin, 2021), a critical-line implementation in pure Python, scales like $\approx n^{3.4}$; the vectorised explicit-inverse baseline of §9.3 like $\approx n^{2.0}$; `cvxcla`'s dense backend like $\approx n^{2.2}$; and its diagonal-plus-low-rank factor backend, solving the identical covariance through the Woodbury identity, like $\approx n^{1.3}$. At $n = 640$ the factor backend is roughly 2460× faster than PyPortfolioOpt and 7.7× faster than the dense backend. The incremental-inverse baseline and the dense backend lie close together, far below PyPortfolioOpt; that gap reflects how PyPortfolioOpt organises its per-step work, not the underlying critical-line algorithm (§9.3). The orange curve reconstructs the frontier with Clarabel (Goulart and Chen, 2024) as n QP solves over a λ-grid; it is drawn here only for context and is discussed in §9.7, not in this section. Markers are the median of 3 repetitions; the shaded band around each spans their min and max.

Bailey and López de Prado (2013) the call `np.linalg.inv(covarF)` sits inside the per-candidate `for i in b` loop that searches for the asset to free; PyPortfolioOpt inherits this structure). `cvxcla` instead forms the affine segment once per step from a single solve and evaluates all event candidates simultaneously (the L matrix of §4.3), so the gap is structural and algorithmic, not a `numpy` artefact.

That still leaves a fair question about `cvxcla`'s own dense backend: it too re-solves from scratch each step, discarding the previous factorisation. Could a CLA that instead *maintains* the free-block inverse and updates it by rank-one formulas do better? We test exactly this with a third implementation (`experiments/inverse_cla.py`): a CLA that shares `cvxcla`'s vectorised event logic but maintains $\Sigma_{\mathcal{F}\mathcal{F}}^{-1}$ through rank-one bordered (asset enters) and deletion (asset leaves) updates, $O(n_{\mathcal{F}}^2)$ per step against the fresh solve's $O(n_{\mathcal{F}}^3)$. It is *not* a reimplementation of PyPortfolioOpt (it avoids PyPortfolioOpt's per-candidate rebuild) and it traces the identical turning points to machine precision (max. weight discrepancy $< 10^{-13}$ across $n = 20$–160), so the comparison against the dense backend is clean: only the linear-algebra strategy differs.

The answer is yes: the incremental-inverse baseline ($\approx n^{2.0}$, 329 ms at $n = 640$) is about 2.4× *faster* than the dense backend ($\approx n^{2.2}$, 792 ms), exactly as the $O(n_{\mathcal{F}}^2)$-versus-$O(n_{\mathcal{F}}^3)$ count predicts. `cvxcla`'s block elimination is therefore not the fastest possible dense strategy, and the package ships the incremental inverse as the opt-in `IncrementalDenseCovariance` backend (§5.2) for callers who

want that win. It is not the *default* because the fresh solve is what composes cleanly with the covariance operator (§5) and is numerically robust step to step: the factor backend never forms $\Sigma_{\mathcal{FF}}$ at all, so there is no inverse to maintain: it solves through the Woodbury identity. The default dense backend gives up the modest incremental-inverse win to keep one turning-point loop that runs over *any* backend, and the factor backend more than repays it: at $n = 640$ it is $\approx 7.7\times$ faster than the dense backend and $\approx 3.2\times$ faster than the incremental-inverse baseline, the only one of the four implementations with a genuinely sub-quadratic exponent ($\approx n^{1.3}$). The structured solve, not the linear-algebra bookkeeping, is the real algorithmic advance.

9.4 Runtime versus factor rank

The advantage of the factor backend is not unconditional: it comes from the low-rank structure, so it must fade as the rank K approaches n. To make this precise we fix the universe at $n = 320$ assets and vary the number of factors $K \in \{20, 40, 80, 160, 320\}$ of the ground-truth covariance, tracing the same frontier with both backends. The dense backend factorises the same 320×320 free block regardless of K, so its time is essentially flat; the Woodbury solve costs $O(n_{\mathcal{F}}K^2 + K^3)$ per step and grows with K. Figure 3 and Table 2 show the crossover: the factor backend is $3.1\times$ faster at $K = 20$, reaches parity near $K = 160$, and is about $2\times$ *slower* at $K = 320$, where the "low-rank" part is full rank and the Woodbury correction is pure overhead. The practical reading is that the structured backend pays off precisely in the regime real risk models inhabit: a few tens of factors over hundreds or thousands of assets ($K \ll n$).

K	`cvxcla` dense [ms]	`cvxcla` factor [ms]	Speedup
20	112.0	36.2	3.1×
40	114.1	43.0	2.7×
80	108.1	59.7	1.8×
160	86.4	78.5	1.1×
320	87.1	194.4	0.4×

Table 2: Frontier trace time at fixed $n = 320$ as the factor rank K varies (median of 3 repetitions). The dense backend is independent of K; the Woodbury backend grows with K and loses its advantage once K is no longer small relative to n. Both return the same ≈ 320-point frontier.

9.5 Exactness against a reference QP solver

The frontier is claimed to be *exact*: each segment (3) is the true optimiser of (1), not an interpolation. We verify this independently. Taking 40 real assets (§9.6, full-history sample covariance, condition number ≈ 200), we trace the frontier with the CLA (29 turning points) and then, at the midpoint λ of every one of the 27 interior segments, re-solve the parametric quadratic program (1) from scratch with a general convex solver (CVXPY/Clarabel (Diamond and Boyd, 2016), tight tolerances) and compare its optimiser against the CLA weights read off the segment by linear interpolation in λ. The agreement is at solver precision (median weight discrepancy 5×10^{-8}, maximum 1.3×10^{-5}) confirming that the affine segments coincide with the QP optima rather than approximating them (`experiments/validate_exact.py`). The midpoint is a representative interior point of each segment; because the segment is affine in λ, agreement at one interior point certifies the whole open segment, and its endpoints are the turning points, which are themselves validated to satisfy the bounds and the constraints (§4.3) as they are recorded.

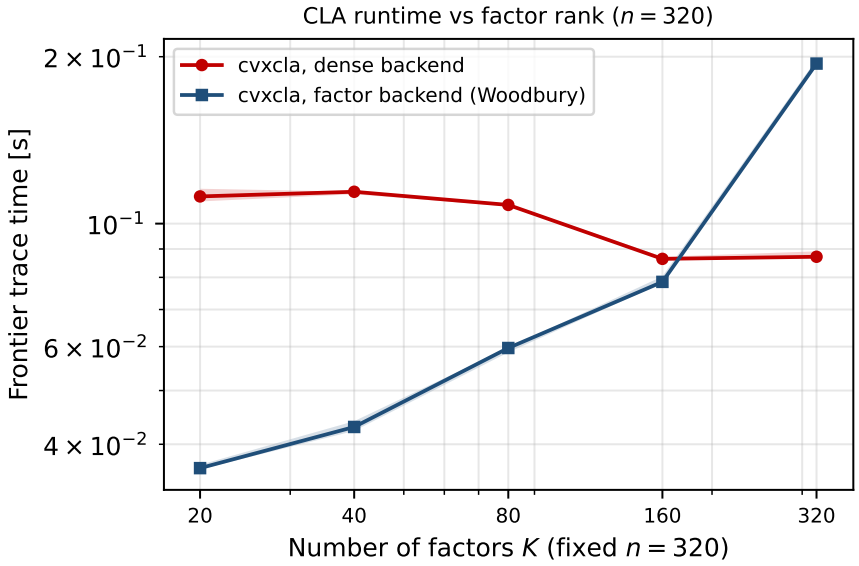

Figure 3: Frontier trace time at fixed $n = 320$ as a function of the factor rank K (log–log). The dense backend is flat in K; the diagonal-plus-low-rank factor backend grows like the Woodbury cost $O(n_{\mathcal{F}}K^2 + K^3)$ and crosses the dense line near $K \approx 160$. The structured backend wins only while $K \ll n$. Markers are the median of 3 repetitions; the shaded band spans their min and max.

The same check extends to general linear constraints, the regime the headline examples do not exercise. On a deterministic 30-asset factor market we trace two constrained frontiers and re-solve every segment midpoint as a general QP carrying the same constraints (`validate_constraints.py`): first a budget together with a characteristic-neutrality row (a two-row equality system, 34 turning points, so the maximum-return vertex is the linear program of §3 rather than the greedy fill), and second a budget with per-sector exposure caps $Gw \le h$ (31 turning points, with an inequality row *active* at 30 of them, so the bordered solve of §10 is genuinely exercised). In both the affine segments match the QP optimiser to solver precision (maximum weight discrepancy $\approx 6 \times 10^{-10}$), confirming that the inequality and multi-row-equality machinery traces the exact constrained frontier, not merely a feasible approximation.

9.6 Empirical example: the S&P 500 frontier

To close, we run the algorithm on real data. The experiment runs off a frozen snapshot shipped with the package, `experiments/data/sp500_pct_returns.parquet` ($N = 494$ assets over $T = 1213$ trading days, 2021-07-30 to 2026-05-29; SHA-256 `b5faa522...9937e`), so the figures below reproduce deterministically and offline, with no dependence on a live data feed. The snapshot was produced once by `experiments/fetch_sp500.py`, which pulls the then-current S&P 500 constituents from Wikipedia, downloads five years of daily adjusted-close prices via `yfinance`, and drops thinly-traded names; that script is retained for provenance and refresh only and is not part of the reproducible path. From the full-history sample mean and covariance (condition number $\approx 1.1 \times 10^5$) the CLA traces the entire long-only frontier (**96 turning points**) in a median of ≈ 14 ms (Figure 4); a rank-20 diagonal-plus-low-rank approximation of the same covariance traces a near-identical frontier (89 turning points) in ≈ 9 ms through the Woodbury backend. The annualised frontier spans expected returns of 0.14–0.89 over volatilities of 0.10–0.87, with a maximum Sharpe ratio of about

2.7. This figure is measured in-sample (the frontier is optimised over the same sample mean and covariance from which it is computed) and so overstates what is achievable out-of-sample; it is reported only to characterise the geometry of the frontier on a realistic problem, not as a claim of attainable performance.

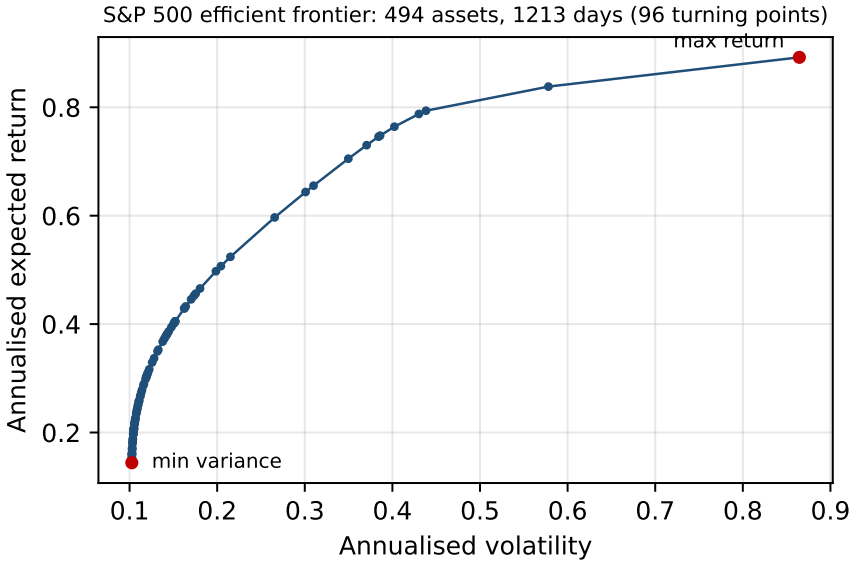

Figure 4: The long-only efficient frontier of 494 S&P 500 constituents from a full-history daily sample covariance ($T = 1213$ days), computed exactly by the CLA. Moments are annualised; each marker is one of the 96 turning points.

A real degeneracy, diagnosed and repaired. The traces above run cleanly because the expected returns are well separated and the covariance is well conditioned. Near-degenerate inputs are more delicate, and earlier versions of the algorithm aborted on them. The cause is now understood, and it is neither a conditioning nor a rank failure in the usual sense. On a near-rank-deficient covariance the smallest eigenvalues span near-flat directions of the objective; accumulated floating-point round-off over the hundreds of turning points of a large trace nudges a free weight a hair (typically 10^{-5} to 10^{-3}) outside its box along one of those directions. The candidate is therefore optimal to solver precision but not exactly feasible, and a strict feasibility check rejects it.

Because the deviation lies in the covariance's flat directions, projecting the candidate back onto the feasible box (and rescaling to restore the budget) changes the objective only at the level of the round-off itself. The implementation does exactly this in `_emit`, and the trace then completes with turning points that remain optimal: across a controlled rank-deficiency sweep every completed point matches a reference QP solve (CVXPY/Clarabel, §9.5) in objective to $\sim 10^{-8}$ (Figure 5), and the full-history frontier of §9.6 is unchanged to the last digit, since well-posed turning points are already strictly feasible and the projection is then a no-op. The short-window S&P 500 covariances that previously aborted now trace to completion.

The repair is deliberately bounded, and the two regimes are told apart by the *conditioning* of the free-asset block, not by the size of the violation it happens to produce. While that block stays numerically full rank its solve is reliable and any infeasibility is round-off, which is projected away; once the free set grows past the covariance rank the block is numerically singular and its

solve is unreliable, so projecting whatever it returns could silently yield a suboptimal frontier. A guard therefore declines as soon as the free block's reciprocal condition number falls below 10^{-12} (the conventional numerical-singularity scale), raising an actionable diagnosis instead. We key the guard on conditioning rather than on the box violation because the violation is the residual of a singular solve and its magnitude varies with the BLAS/LAPACK build, whereas the conditioning is read from the block's symmetric eigenvalues and is identical on every platform. This is the precise guarantee *for rank and conditioning degeneracy*: while the free-asset block stays full rank the trace completes and every turning point is optimal; when it does not, the algorithm declines rather than mislead. It speaks to the linear algebra, not to event ties: the distinct, combinatorial degeneracy of many events coincident at one $\lambda^\star$ is handled by the Bland-style tie-break of §4.4, which is robust on the degenerate problems we test but, like the worst-case turning-point count, is not proven to cover every adversarial tie-heavy or very large instance. A well-conditioned, full-rank estimate (ample history, or a structured `FactorCovariance`, which is positive definite by construction) stays in the first regime. The experiment is reproducible from `experiments/degeneracy_boundary.py`.

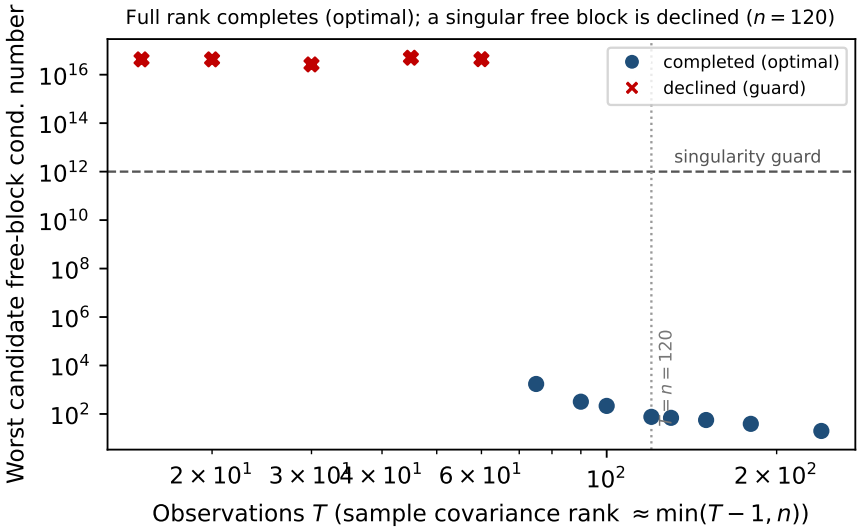

Figure 5: A controlled rank-deficiency sweep at $n = 120$: the sample covariance is estimated from T observations, so its rank is $\approx \min(T-1, n)$ and falls as T shrinks (leftward). Each marker is the worst conditioning (2-norm condition number) of a *candidate* turning point's free-asset block. Blue circles complete the full trace, and every completed frontier is optimal, matching a reference QP in objective to $\sim 10^{-8}$. Red crosses are declined. Completions sit far below the singularity guard (dashed, condition number 10^{12}): their free blocks stay numerically full rank (condition number $\lesssim 10^3$), so the only infeasibility is round-off in the covariance's near-flat directions, repaired by projecting onto the feasible box. Declines sit far above it (condition number $\sim 10^{16}$): the free set has grown past the covariance rank, the block is numerically singular and its solve is unreliable, so the algorithm declines rather than risk a suboptimal frontier. The boundary is monotone in T, and the wide gap between the two regimes leaves the guard threshold uncritical.

The combinatorial axis: a tie-heavy stress test. The guarantee above is for rank and conditioning; the orthogonal, combinatorial axis is event ties, handled by the Bland-style rule of §4.4. We characterise it empirically rather than leave it asserted (`tie_degeneracy.py`): across four de-

liberately degenerate families and 128 instances (n up to 240), exactly-tied means and exact asset duplicates both trace to completion (64/64) – the duplicates included because the tie-break never frees both copies of a pair, so the singular covariance never reaches the free block; a constraint-heavy family of $p \approx n/3$ group caps also completes (32/32) with as many as 53 inequality rows active at a single turning point; and an over-constrained family of $p = n$ overlapping caps is *declined* at the maximum-return vertex (32/32), the documented degenerate-vertex boundary, rather than mis-solved. No instance reached the iteration cap, and every completed trace used at most 1.3% of it. This is empirical reassurance on the combinatorial axis, not the worst-case proof the rank/conditioning guard provides, and constructing an adversarial cycling instance remains open.

9.7 A general-solver baseline: a QP swept over λ

The comparisons so far are against other critical-line implementations. The complementary question is how the exact trace compares to the route a general convex-optimisation user would actually take: hand the quadratic program (1) at a fixed λ to a solver and sweep λ to recover the frontier (§8). We make this concrete with Clarabel (Goulart and Chen, 2024), the interior-point conic solver that CVXPY selects by default, driven through its *native* Python API with no modelling layer in between, so the baseline carries no per-solve canonicalisation overhead and the comparison is as favourable to it as possible. For each universe we solve (1) at n values of λ spanning the frontier (a grid as fine as the universe is large) and time the whole sweep against a single `cvxcla` trace (`experiments/clarabel_baseline.py`).

Even with the modelling layer stripped away, the grid is slower at every size and the gap widens with n: from $\approx 1.7\times$ at $n = 20$ to $\approx 59\times$ at $n = 320$, where one `cvxcla` trace takes ≈ 0.12 s and the 320 Clarabel solves take ≈ 7.4 s (≈ 23 ms each). The sweep grows like $\approx n^{2.9}$ – n solves, each an interior-point QP whose own cost climbs with n – far steeper than the trace (Figure 6). And the grid only *samples* the frontier: each Clarabel solution matches the exact piecewise-linear frontier (read off `cvxcla`'s turning points by interpolation in λ, an independent exactness check alongside §9.5) to the solver's tolerance, a maximum weight discrepancy of $\approx 4 \times 10^{-3}$ at Clarabel's default settings, whereas the trace returns the frontier exactly and in full. The reading is the one of §8: a general solver is the right tool for a *single* risk target, but reconstructing the whole frontier by gridding is both slower and only approximate.

10 Connection to homotopy methods: least angle regression and the LASSO

The CLA is one instance of a broader idea that recurs across optimisation and statistics: when the optimiser of a convex problem is followed as a single scalar parameter varies, the solution path is frequently piecewise linear, and it can be traced exactly by stepping from one breakpoint to the next while maintaining an active set. The same structure drives least angle regression (LARS) and the LASSO homotopy in statistical learning (Tibshirani, 1996; Osborne et al., 2000; Efron et al., 2004). Making the correspondence explicit places the CLA in a family it is seldom presented alongside, and separates the features that are essential to the method from those particular to the portfolio setting.

The most direct relative is in control rather than statistics. The CLA is a one-parameter *multi-parametric quadratic program* (mpQP): a strictly convex QP whose linear term depends affinely on a parameter, and whose solution map is therefore piecewise affine over a polyhedral partition of the parameter space. This is the structure exploited by explicit model predictive control, where

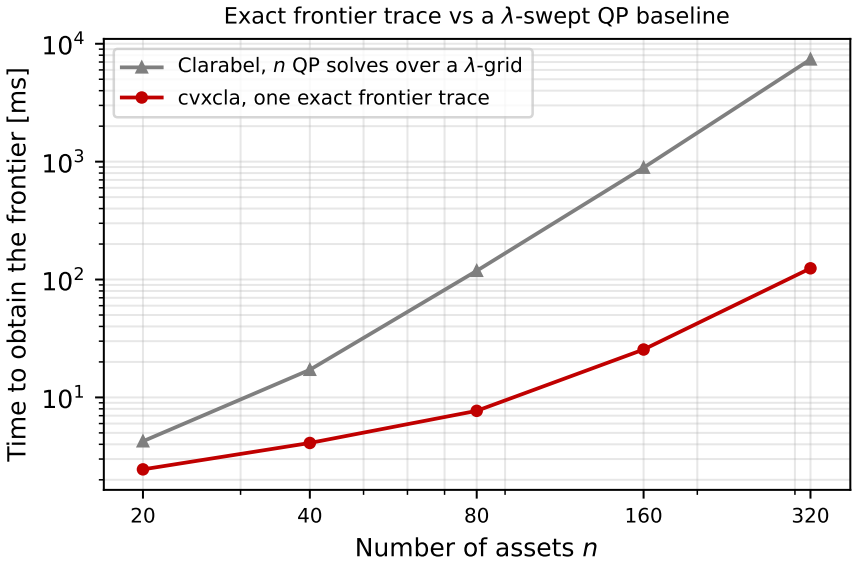

Figure 6: Time to obtain the frontier as a function of the number of assets n (log–log): a single exact `cvxcla` trace against reconstructing the frontier with n Clarabel (Goulart and Chen, 2024) QP solves over a λ-grid (the solver driven natively, no CVXPY layer). The grid sweep grows like $\approx n^{2.9}$ and is $\approx 59\times$ slower than the trace at $n = 320$, and yields only an n-point sampling rather than the exact piecewise-linear frontier.

the optimal control law is precomputed as a piecewise-affine function of the state by mpQP path-following (Bemporad et al., 2002; Tøndel et al., 2003). The turning points of the CLA are the breakpoints of that map for the single risk-aversion parameter, and the event logic of §4.3 is the one-dimensional specialisation of the critical-region traversal used there. The general mpQP machinery also handles arbitrary polyhedral constraints, and `cvxcla` admits them too, through the inequality rows $Gw \leq h$ of (1). The mechanism is the row analogue of the box active set. A box bound is a per-variable inequality: an asset is free or blocked, and the events are a free weight reaching a bound or a blocked multiplier releasing it. A row of $Gw \leq h$ is a per-row inequality: it is active (held at $g_i^{\mathsf{T}} w = h_i$) or slack, and the events are a slack row's residual reaching zero, so it activates, or an active row's multiplier reaching zero, so it releases. Both kinds of event are the same "smallest positive step to the next breakpoint" along the affine segment; the row events are simply scanned over the p rows of G in addition to the n box coordinates (§4.3).

Crucially, this does *not* sacrifice the structure that the equality-only case enjoys. An active inequality row enters the reduced KKT system (2) exactly as an equality row does: stacking $C = [A; G_{\mathcal{S}}]$ over the active rows recovers the equality-only solve with A replaced by C, so each step is still a solve against the free covariance block $\Sigma_{\mathcal{F}\mathcal{F}}$ (through the operator interface of §5, never an $n \times n$ matrix) feeding an $(m + |\mathcal{S}|) \times (m + |\mathcal{S}|)$ Schur complement. The box bounds remain a separate, cheaper per-variable active set rather than being folded into G, so the common box-only problem pays nothing for the generalisation.

What does *not* work, and is worth recording because it is the natural first reflex, is turning an inequality into an equality with a slack. Writing $g_i^{\mathsf{T}} w + s_i = h_i$ with $s_i \geq 0$ leaves the slack with no curvature in the objective, so the augmented Hessian $\mathrm{diag}(\Sigma, 0)$ is only positive *semi*-definite and the reduced solve is singular whenever a slack is free (precisely when its inequality is inactive).

Admitting general inequalities therefore goes through the bordered system above and a per-row event, not a change of variables, which is exactly what the implementation does.

The LASSO solves, for a response y and design matrix X,

$$\min_{\beta} \ \tfrac{1}{2}\,\|y - X\beta\|_2^2 \ + \ \lambda\,\|\beta\|_1,$$

and its minimiser $\beta(\lambda)$ is a continuous, piecewise-linear function of the penalty λ (Efron et al., 2004; Rosset and Zhu, 2007). As λ decreases from the value at which $\beta = 0$ towards 0, the path proceeds in straight segments; at each breakpoint the active set (the support of β) changes, either because an inactive variable whose correlation with the current residual reaches λ enters, or because an active coefficient reaches zero and leaves. LARS computes these breakpoints and segment directions directly, exactly as the CLA computes turning points and the affine segments (3) between them.

The two algorithms line up term for term (Table 3).

Ingredient	Critical Line Algorithm	LARS / LASSO homotopy
Swept parameter	risk aversion $\lambda : \infty \to 0$	penalty $\lambda : \infty \to 0$
Solution path	$w(\lambda) = \alpha + \lambda\beta$ (pw. linear)	$\beta(\lambda)$ (pw. linear)
Active set	free set $\mathcal{F}$ (weights off the bounds)	support (nonzero β_j)
Linear solve	reduced KKT over $\Sigma_{\mathcal{F}\mathcal{F}}$	least squares over $X_{\mathcal{A}}$
Enter event	blocked multiplier changes sign	residual correlation reaches λ
Leave event	free weight reaches a box bound	active coefficient reaches zero
Step length	smallest positive step to the next breakpoint	
Degeneracy	Bland lowest-index tie-break (§4.4)	tie-break among simultaneous hits

Table 3: The Critical Line Algorithm and the LARS/LASSO homotopy as two instances of the same parametric active-set scheme. The role played by the covariance Σ and the mean μ in the reduced KKT solve (4) is played by the Gram matrix $X^{\mathsf{T}}X$ and the vector $X^{\mathsf{T}}y$ in the regression path.

The correspondence is not a coincidence. Rosset and Zhu (2007) characterise when a regularised solution path is piecewise linear: a quadratic (or piecewise-quadratic) loss together with a polyhedral, i.e. piecewise-linear, constraint or penalty suffices. Both problems meet the criterion. The CLA minimises a quadratic objective (1) over a polyhedral feasible set (box bounds, linear equalities, and linear inequalities); the LASSO minimises a quadratic loss under a polyhedral (ℓ_1) penalty. In each case the KKT conditions are affine in the parameter, the system matrix is constant while the active set is fixed, and the active set can change only at the discrete parameter values that the homotopy steps between. The "smallest positive step to the next breakpoint" rule of §4.3 is the same step-length computation that advances the LARS path, and both inherit it, for the same anti-cycling reasons, from the simplex method (§4.4).

Three differences are worth stating precisely, since they are exactly what distinguishes the portfolio problem.

What induces the active set. In the CLA the active set is carved out by *explicit* polyhedral constraints: an asset is blocked when it sits on a box bound, and the events are free weights reaching those bounds, or blocked multipliers releasing them. In the LASSO the active set is *induced* by the ℓ_1 penalty through its subgradient: sparsity is an emergent property of the geometry, not a constraint the modeller writes down. The same polyhedral mechanism runs in opposite directions, one imposing the faces and the other discovering them.

Entering and leaving. Forward LARS only ever *adds* variables to the active set; the LASSO modification restores the ability to drop one when its coefficient crosses zero. The CLA needs both moves throughout, as assets become free and blocked while λ decreases, so it corresponds to the full LASSO homotopy rather than to forward LARS.

The data enter only through linear algebra. The Gram matrix $X^\mathsf{T}X$ that defines the LARS direction is played here by the covariance Σ in the reduced KKT solve (4), and $X^\mathsf{T}y$ by the expected-return vector μ. Seen this way, the covariance-operator abstraction of §5 is the statement that the path-tracing logic touches Σ only through a few linear-algebraic operations, just as the LARS recursion touches X only through Gram-matrix solves; a structured Σ (a factor model) is the portfolio analogue of a structured or kernelised design. The package makes the correspondence literal: the `GramCovariance` backend (§5.4) builds Σ from the centred data matrix X_c directly through (13), so the CLA runs off X_c exactly as LARS runs off the design matrix, and the protocol it calls is named `QuadraticForm` rather than after the covariance for this reason.

The bridge is more than analogy. Brodie et al. (2009) add a gross-exposure constraint $\|w\|_1 \le c$ to the Markowitz problem and show that the result *is* a LASSO, with c promoting sparse, stable portfolios. For the long-only, fully-invested problem this constraint is silent, since the budget already fixes $\|w\|_1 = \mathbf{1}^\mathsf{T}w = 1$, which is why the plain CLA traces a frontier rather than a sparsity path. Once short positions are admitted under a gross-exposure cap, however, the parametric program the CLA traces and the LASSO path coincide: the two are then not merely structurally similar but two homotopies over the same parametric quadratic program.

10.1 A LASSO path through the same engine

The correspondence is not only conceptual: the package makes it executable. The class `Lasso` (`src/cvxcla/lasso.py`) is driven by the *same* `cvxcla.pathtracer.trace` loop and the same Bland event selection as the `CLA`; only the segment solve and the meaning of an event differ. It supplies the Gram matrix $X^\mathsf{T}X$ (as a `DenseCovariance`, the backend named `QuadraticForm` precisely because it is not specific to a covariance) and the linear data $X^\mathsf{T}y$, and traces the entire LASSO regularisation path from $\lambda_{\max} = \|X^\mathsf{T}y\|_\infty$ (where $\beta = 0$) down to the least-squares fit at $\lambda = 0$. As a check that this is the *same* homotopy the reference statistical-learning solvers trace, we compare it breakpoint by breakpoint against scikit-learn's `lars_path` (Pedregosa et al., 2011) on a standardised 60×12 regression (Listing 6); the two paths coincide to solver precision (maximum coefficient discrepancy $\approx 3 \times 10^{-15}$ over all 13 breakpoints, Figure 7). The experiment is reproducible from `experiments/lasso_path.py`.

Listing 6: The CLA path tracer drives a LASSO through `cvxcla.Lasso`, matched against scikit-learn's `lars_path`.

```
import numpy as np
from cvxcla import Lasso
from sklearn.linear_model import lars_path

# standardised design X (m x n), centred response y (see experiments/lasso_path.py)
lasso = Lasso(x=X, y=y) # the same trace() loop as the CLA
alphas, _active, coefs = lars_path(X, y, method="lasso")

# scikit-learn reports alphas in the 1/(2m) loss convention; lam = m * alpha
m = X.shape[0]
diffs = [np.max(np.abs(lasso.solution(m * a) - coefs[:, i]))
```

```
        for i, a in enumerate(alphas)]
print("breakpoints (cvxcla, sklearn):", len(lasso.path), len(alphas))
print("max |beta_cvxcla - beta_sklearn|:", f"{max(diffs):.1e}")
```

```
breakpoints (cvxcla, sklearn): 13 13
max |beta_cvxcla - beta_sklearn|: 3.2e-15
```

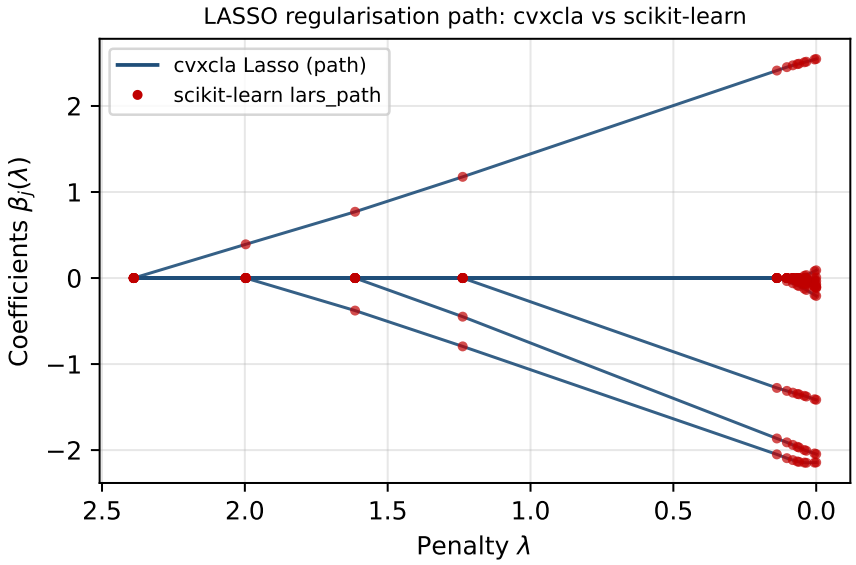

Figure 7: The LASSO regularisation path of a standardised 60×12 regression, traced by `cvxcla.Lasso` through the same parametric active-set engine as the efficient frontier. Each line is one coefficient $\beta_j(\lambda)$ as the penalty λ decreases from $\lambda_{\max}$ (right) to 0 (left); the markers are the breakpoints reported independently by scikit-learn's `lars_path`. The two implementations agree to solver precision ($\approx 3 \times 10^{-15}$), confirming that the CLA engine and the LARS/LASSO homotopy trace the identical path.

10.2 A constrained LASSO through the same engine

The unification is not confined to the textbook LASSO. The same general linear inequality constraints the CLA carries, $G\beta \leq h$, drop into the LASSO path unchanged: an active row enters the reduced system as the bordered Schur complement of §4.1, and the generalised correlation that drives the enter/leave events simply carries the active-row multipliers, $X^\mathsf{T}(y - X\beta(\lambda)) - G_\mathcal{S}^\mathsf{T}\eta(\lambda)$. The path stays piecewise linear (a quadratic loss under a polyhedral penalty *and* polyhedral constraints, the criterion of Rosset and Zhu (2007)), and `cvxcla.Lasso` accepts G, h exactly as the `CLA` does. We require $h > 0$ so the path can start from the feasible $\beta = 0$ with every row slack, the same first vertex as the unconstrained LASSO; an equality system $A\beta = b$, or a zero entry of h, would need the linear-programming feasibility seed of §3 and is left to future work. On a standardised 60×12 regression with per-group exposure caps $G\beta \leq h$ (15 breakpoints, a cap active at 9 of the 14 interior segments), every segment midpoint matches a per-λ constrained QP solve (CVXPY/Clarabel) to $\approx 2 \times 10^{-10}$ in coefficients and to machine precision in feasibility (`validate_lasso_constraints.py`), so the one path-tracing engine returns the exact *constrained* regularisation path as well as the exact constrained frontier.

The sign restriction $\beta \geq 0$ (the non-negative LASSO) is where the unification is most literal. With $\beta \geq 0$ the penalty $\|\beta\|_1 = \mathbf{1}^\mathsf{T}\beta$ becomes *linear*, so the problem is exactly the CLA's box-bounded parametric quadratic program: Hessian $H = X^\mathsf{T}X$, lower bound 0, and the λ-linear term $X^\mathsf{T}y - \lambda\mathbf{1}$. The homotopy is then the standard path restricted to positive signs (only the $+\lambda$ entry event, started from $\beta = 0$ at $\lambda_{\max} = \max_j (X^\mathsf{T}y)_j$). `cvxcla` traces it with `nonneg=True`, and every segment midpoint matches the per-λ non-negative QP to $\approx 3 \times 10^{-11}$ (`validate_lasso_constraints.py`). The fluent builder of §7 carries over unchanged for both, so a constrained or non-negative path reads the same way as the constrained frontier (Listing 7).

Listing 7: The constrained LASSO through the same fluent builder as the CLA: `.inequality(G, h)` then `.trace()`.

```
from cvxcla import Lasso

# per-group exposure caps G beta <= h on a standardised regression
lasso = Lasso.problem(X, y).inequality(G, h).trace()
print("breakpoints:", len(lasso.path))

# the non-negative LASSO (beta >= 0) is the same builder with one more step
nonneg = Lasso.problem(X, y).non_negative().trace()
```

11 Conclusion

The Critical Line Algorithm computes the complete mean–variance efficient frontier by recognising its piecewise-linear structure in weight space and walking from the maximum-return portfolio to the minimum-variance portfolio, flipping one asset between the free and blocked sets at each turning point. The `cvxcla` implementation realises each step as a single block-eliminated KKT solve, hardens the event logic against degeneracy and floating-point noise, and abstracts the covariance behind a small operator protocol so that structured risk models obtain the same exact frontier without ever forming the dense matrix, and at a fraction of its cost when their structure is genuinely low-rank. The same parametric active-set mechanism places the CLA alongside multi-parametric quadratic programming and the LARS/LASSO homotopy: the operator interface is what lets one path-tracing engine serve both the portfolio frontier and, with the covariance replaced by a Gram matrix, a LASSO regularisation path.

The Critical Line Algorithm is Markowitz's, and this paper claims none of it as new. Seven decades on, it remains the only method that computes the entire mean–variance frontier exactly rather than sampling it, and our aim has been to give it a modern, open, and structured home rather than to improve upon it. The work is, in that sense, a bow to Harry Markowitz (1927–2023), who gave the field both the question and its constructive answer.

Acknowledgements

The author thanks Stephen Boyd and Philipp Schiele. The first versions of the solver were implemented while the author stayed for a sabbatical at Boyd's group at Stanford University.

References

D. H. Bailey and M. López de Prado. An open-source implementation of the critical-line algorithm for portfolio optimization. *Algorithms*, 6(1):169–196, 2013.

A. Bemporad, M. Morari, V. Dua, and E. N. Pistikopoulos. The explicit linear quadratic regulator for constrained systems. *Automatica*, 38(1):3–20, 2002.

J. Brodie, I. Daubechies, C. De Mol, D. Giannone, and I. Loris. Sparse and stable Markowitz portfolios. *Proceedings of the National Academy of Sciences*, 106(30):12267–12272, 2009.

D. Cajas. Riskfolio-Lib: portfolio optimization and quantitative strategic asset allocation in Python. https://github.com/dcajasn/Riskfolio-Lib, 2021.

S. Diamond and S. Boyd. CVXPY: a Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016.

B. Efron, T. Hastie, I. Johnstone, and R. Tibshirani. Least angle regression. *The Annals of Statistics*, 32(2):407–499, 2004.

A. Fu, B. Narasimhan, and S. Boyd. CVXR: an R package for disciplined convex optimization. *Journal of Statistical Software*, 94(14):1–34, 2020.

P. J. Goulart and Y. Chen. Clarabel: An interior-point solver for conic programs with quadratic objectives. arXiv:2405.12762, 2024.

M. Hirschberger, Y. Qi, and R. E. Steuer. Large-scale MV efficient frontier computation via a procedure of parametric quadratic programming. *European Journal of Operational Research*, 204(3):581–588, 2010.

H. M. Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.

H. M. Markowitz. The optimization of a quadratic function subject to linear constraints. *Naval Research Logistics Quarterly*, 3(1–2):111–133, 1956.

H. M. Markowitz. *Portfolio Selection: Efficient Diversification of Investments*. John Wiley & Sons, New York, 1959.

H. M. Markowitz and G. P. Todd. *Mean-Variance Analysis in Portfolio Choice and Capital Markets*. Frank J. Fabozzi Associates, New Hope, PA, 2000.

H. Markowitz, D. Starer, H. Fram, and S. Gerber. Avoiding the downside: A practical review of the critical line algorithm for mean–semivariance portfolio optimization. In *Handbook of Applied Investment Research*, chapter 17. World Scientific, 2021. https://doi.org/10.1142/9789811222634_0017.

R. A. Martin. PyPortfolioOpt: portfolio optimization in Python. *Journal of Open Source Software*, 6(61):3066, 2021. https://doi.org/10.21105/joss.03066.

C. Nicolini, M. Manzi, and H. Delatte. skfolio: Portfolio optimization in Python. arXiv:2507.04176, 2025.

A. Niedermayer and D. Niedermayer. Applying Markowitz's critical line algorithm. In *Handbook of Portfolio Construction*, pages 383–400. Springer, 2010.

M. R. Osborne, B. Presnell, and B. A. Turlach. A new approach to variable selection in least squares problems. *IMA Journal of Numerical Analysis*, 20(3):389–403, 2000.

F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

A. F. Perold. Large-scale portfolio optimization. *Management Science*, 30(10):1143–1160, 1984.

S. Rosset and J. Zhu. Piecewise linear regularized solution paths. *The Annals of Statistics*, 35(3):1012–1030, 2007.

W. F. Sharpe. A simplified model for portfolio analysis. *Management Science*, 9(2):277–293, 1963.

M. Stein, J. Branke, and H. Schmeck. Efficient implementation of an active set algorithm for large-scale portfolio selection. *Computers & Operations Research*, 35(12):3945–3961, 2008.

R. Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society, Series B*, 58(1):267–288, 1996.

P. Tøndel, T. A. Johansen, and A. Bemporad. An algorithm for multi-parametric quadratic programming and explicit MPC solutions. *Automatica*, 39(3):489–497, 2003.

P. Wolfe. The simplex method for quadratic programming. *Econometrica*, 27(3):382–398, 1959.

D. Würtz, Y. Chalabi, W. Chen, and A. Ellis. *Portfolio Optimization with R/Rmetrics*. Rmetrics Association and Finance Online, 2009. R package `fPortfolio`.